MARMARA UNIVERSITY

FACULTY OF ENGINEERING

CSE3055-DATABASE SYSTEMS

(Fall 2025)

Step-III

Due: 26.12.2025/23.59

Department	Student ID	Name & Surname
Computer Engineering	150123852	İlayda İlhan
Computer Engineering	150123035	Canan Pazarbaşı
Computer Engineering	150122050	Ömer Ercan Dayan

Table Of Contents

1.2. Scope and Purpose 3
1.3. Business Context & ER Model Alignment 4
1.4. Data Integrity and Compliance 4
2.1. Entity Analysis 4
2.2. Business Rules and Constraints Implementation 5
3.2. Technical Implementation & Constraints Compliance 6
3.3. Database Tables 8
3.4. Views 15
3.5. Stored Procedures 19
I. Data Manipulation Procedures 19
II. Business Logic Procedures 31
III. Reporting Procedures 37
3.6. Triggers 41
4.Evolution from Conceptual Design to Physical Implementation 43
5. Conclusion 44

1. Project Description

1.1. Project Overview

This database project aims to design and implement a robust relational database management system for Dene Technology, a company in the embedded systems sector. The primary objective is to transition the company from manual, disparate record-keeping to a centralized, digital architecture.

The system is engineered to digitize the entire operational workflow of the Operations and Production Department. It facilitates the systematic management of the supply chain, production lifecycle, quality assurance protocols, and financial transactions. By integrating data from various departments—such as inventory, sales, and operations—the system ensures data consistency, reduces operational redundancy, and provides actionable insights through advanced reporting views and stored procedures.

1.2. Scope and Purpose

The project scope is defined to cover end-to-end business processes, categorized into the following modules:

- **Customer Relationship Management:** Records comprehensive customer profiles (individual and corporate), tracks historical orders, and manages contact information to facilitate accurate invoicing.
- **Production & Inventory Management:** Tracks the lifecycle of products, categorizing them by manufacturing origin (in-house vs. outsourced). It monitors stock levels in real-time and links products to specific production machines.
- **Supply Chain & Partnerships:** Manages a database of external suppliers for raw materials and partners for joint ventures, tracking performance ratings and partnership history.
- **Asset Management:** Logs the operational status (Active, Inactive, Under Maintenance) of production machinery. It enforces maintenance schedules automatically triggered by quality control failures.
- **Quality Assurance:** Documents rigorous inspection results for every manufactured item. It links quality outcomes to specific employees and machines, enabling performance auditing.
- **Financial Operations:** Manages the generation of invoices with automated tax calculations and tracks incoming/outgoing payments with strict approval workflows.

Out of Scope:

- Real-time telemetry and sensor data processing from IoT devices.

- Complex ERP algorithms for predictive demand forecasting.
- Detailed Human Resources modules such as payroll processing, shift scheduling, or performance reviews.

1.3. Business Context & ER Model Alignment

The ER diagram confirms that:

- Each **product** is linked to at least one **customer**, **employee**, **machine**, and **quality check**.
- **Invoices** and **payments** are strictly tied to products and involve customers, suppliers, or partners.
- **Suppliers** and **partners** are connected through employees and have direct payment and invoicing relationships.
- **Machines** are dedicated to specific products, ensuring traceability in production.
- **Quality checks** are performed by employees and associated with products.

This structure ensures data integrity, supports business rules, and enables end-to-end process visibility.

1.4. Data Integrity and Compliance

During the project's initial phase, it was clarified that **no personal or sensitive data** beyond professional contact information is stored. The database complies with operational data standards and focuses solely on business process management.

2. Data and Requirements Analysis

The database is designed to support the core entities required for the production lifecycle. Key entities include **Customer** (placing orders), **Employee** (managing operations), **Product** (items manufactured or supplied), **Machine** (production assets), and **Supplier/Partner** (external collaborators). Business rules dictate strict relationships; for example, every product must undergo a **Quality Check**, and all financial transactions (**Payment**, **Invoice**) must be approved by authorized personnel. The system enforces data integrity to ensure that production cannot proceed without defined resources (machines/employees).

2.1. Entity Analysis

The system is built around **nine core entities**, each representing a critical component of Dene Technology's business operations:

1. **Customer:** Represents the entities purchasing products. The system distinguishes between 'Individual' and 'Corporate' customers via the CustomerType attribute to tailor communication and invoicing requirements. Unique constraints on the Email field prevent duplicate registrations.
2. **Employee:** Stores data on personnel involved in operations. A self-referencing relationship (SupervisorID) is implemented to model the organizational hierarchy (e.g., Technicians reporting to Managers).
3. **Product:** The central entity of the system. It supports a dual-sourcing model: products can be linked to a MachineID (manufactured in-house) or a SupplierID (outsourced). It includes dynamic stock management and cost tracking.
4. **Machine:** Tracks production assets. Crucially, this entity includes a Status attribute constrained to specific states ('Active', 'UnderMaintenance'). It interacts with the **QualityCheck** entity via triggers to automate maintenance scheduling upon failure detection.
5. **Supplier & Partner:** These entities manage external stakeholders. Supplier tracks raw material sources and ratings, while Partner manages joint-production collaborators. Both are linked to financial transactions.
6. **Invoice:** Represents the legal demand for payment. It utilizes **Computed Columns** to automatically calculate TotalWithTax based on the base amount and prevailing TaxRate, ensuring financial accuracy without manual error.
7. **Payment:** Tracks the actual transfer of funds. It is strictly linked to an Invoice to prevent orphaned financial records and includes approval fields (ApprovedBy) for audit trails.
8. **QualityCheck:** Acts as a gatekeeper in the production process. It records the Result ('Pass'/'Fail') and references the specific Machine and Employee involved, providing granular traceability for defects.

2.2. Business Rules and Constraints Implementation

To ensure the system reflects real-world constraints, the following business rules have been enforced at the database level:

1. **Maintenance Automation:** If a product fails a quality check (Result = 'Fail'), the machine used for that product is automatically flagged as 'UnderMaintenance' via the Trigger TR_QC_FAIL_UPDATE_MACHINE, preventing further production of defective goods.
2. **Financial Integrity:** A customer cannot be deleted from the system if they have active debt or linked financial records. This is managed via the SP_DELETE_CUSTOMER stored procedure, which ensures safe, cascading deletion via a Transaction block.
3. **Stock Control:** Product stock levels cannot fall below zero. This is enforced through logic in the SP_UPDATE_PRODUCT_STOCK procedure which raises a custom error if negative values are attempted.

4. **Unique Identification:** Critical fields such as Customer.Email are enforced with **UNIQUE** constraints to prevent duplicate registrations.
5. **Mandatory Associations:** Every production record must be associated with a valid Machine and Employee, enforced via **Foreign Key** constraints and validation logic in SP_ADD_PRODUCT.

3.Database Design

3.1. Diagram of Whole Database

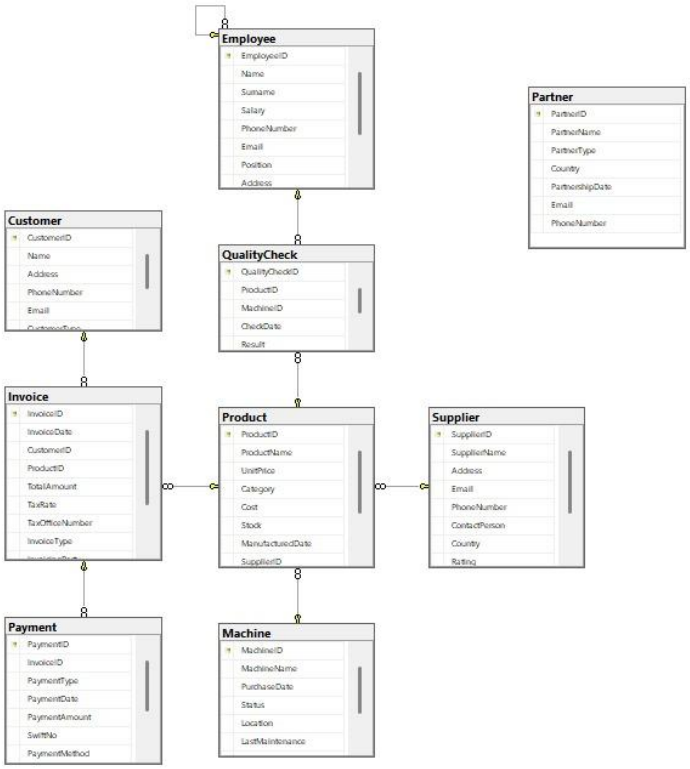

Figure 1. *Database diagram taken from the database diagram tab.*

3.2. Technical Implementation & Constraints Compliance

In accordance with the project requirements, the database has been implemented with strict adherence to normalization rules and SQL Server features to ensure performance and integrity.

i. Normalization (3NF) All 9 tables have been normalized to the **Third Normal Form (3NF)**.

- **1NF:** All columns contain atomic values.
- **2NF:** All non-key attributes are fully dependent on the Primary Key.

- **3NF:** There are no transitive dependencies (e.g., Supplier details are stored in a separate table, referenced by SupplierID in the Product table).

ii. Data Types Appropriate data types were chosen for optimal storage and precision:

- **DECIMAL(12,2)**: Used for monetary values (Price, Cost, Salary) to avoid floating-point errors.
- **NVARCHAR**: Used for text fields to support international characters.
- **INT & IDENTITY**: Used for Primary Keys.
- **DATE**: Used for fields like HiredDate where time component is not required.

iii. Data Population The database has been populated with a substantial dataset to simulate real-world usage.

- **Volume:** Each table contains approximately **25 records**.
- **Diversity:** The data includes various scenarios (e.g., Active/Inactive machines, Pass/Fail quality checks).

iv. Indexes Specific non-clustered indexes were created to improve query performance:

- **IX_Product_Category:** Optimizes filtering products by category.
- **IX_Payment_Date:** Optimizes financial reporting queries based on date ranges.

v. Unique Constraints To prevent data duplication:

- **Customer.Email:** Ensures that every customer has a unique email address.

vi. Identity Columns All 9 tables use the IDENTITY(1,1) property for their Primary Keys to auto-generate unique sequential IDs.

vii. Check Constraints Domain integrity is enforced using Check Constraints:

- **CK_Machine_Status:** Restricts status to 'Active', 'UnderMaintenance', or 'Inactive'.
- **CK_QC_Result:** Ensures the result is only 'Pass' or 'Fail'.
- **CK_Payment_Amount:** Prevents negative payment amounts.

viii. Default Constraints Default values are provided for optional fields:

- **Product.Stock:** Defaults to 0.
- **Machine.Status:** Defaults to 'Active'.
- **Invoice.TaxRate:** Defaults to 20.0.

ix. Computed Columns Dynamic calculations are stored using computed columns:

- **Invoice.TotalWithTax:** Automatically calculates TotalAmount * (1 + TaxRate / 100.0).

3.3. Database Tables

The database's physical design includes 9 tables, conforming to the 3rd Normal Form (3NF) principles. Primary Key, Foreign Key, Check Constraint, Default Value, and Computed Column properties were used to ensure data integrity when creating the tables.

1. Customer Table

- **Definition:** Stores individual and corporate customer information used for order and invoicing processes.
- **Primary Key:** CustomerID (INT, Identity 1,1)
- **Constraints:**
  - **Unique:** Email column ensures no duplicate customer registrations.
  - **Default:** RegistrationDate defaults to the current system date (GETDATE()).
- **Columns:** Name, Address, PhoneNumber, CustomerType.

CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
1	Ali Yılmaz	İstanbul	05001112233	ali.yilmaz@exa...	Bireysel	2025-12-24
2	Ayşe Demir	Ankara	05003334455	ayse.demir@ex...	Kurumsal	2025-12-24
3	Mehmet Kaya	İzmir	05001234567	mehmet.kaya@...	Bireysel	2025-12-24
4	Elif Koç	Bursa	05007894561	elif.koc@exam...	Bireysel	2025-12-24
5	Canan Aydın	Bingöl	05009283746	canan.aydin@e...	Kurumsal	2025-12-24
6	Burak Şahin	Konya	05008887766	burak.sahin@e...	Bireysel	2025-12-24
7	Zeynep Arslan	Eskişehir	05009990012	zeynep.arslan@...	Kurumsal	2025-12-24
8	Fatma Çelik	Kayseri	05006665544	fatma.celik@ex...	Bireysel	2025-12-24
9	Ahmet Öz	Gaziantep	05007774422	ahmet.oz@exa...	Kurumsal	2025-12-24
10	Selin Er	Mersin	05004561239	selin.er@exam...	Bireysel	2025-12-24
11	Hülya Kaplan	Tekirdağ	05009384756	hulya.kaplan@...	Kurumsal	2025-12-24
12	Orhan Bulut	Muğla	05002568974	orhan.bulut@e...	Bireysel	2025-12-24
13	Deniz Taş	Trabzon	05003847562	deniz.tas@exa...	Kurumsal	2025-12-24
14	Gamze Kar	Balıkesir	05007452319	gamze.kar@exa...	Bireysel	2025-12-24
15	Murat Uzun	Adana	05006123478	murat.uzun@e...	Kurumsal	2025-12-24
16	Aslı Tunç	Edirne	05003457812	asli.tunc@exa...	Bireysel	2025-12-24
17	Kerem Gök	Rize	05009234581	kerem.gok@ex...	Kurumsal	2025-12-24
18	Nazlı Kır	Aydın	05004321234	nazli.kir@exam...	Kurumsal	2025-12-24
19	Uğur Ateş	Van	05006549871	ugur.ates@exa...	Bireysel	2025-12-24
20	Sevgi Baran	Malatya	05007891234	sevgi.baran@ex...	Kurumsal	2025-12-24
21	Tolga Cenk	Kütahya	05005678123	tolga.cenk@ex...	Bireysel	2025-12-24
22	Melis Gür	Sakarya	05009998877	melis.gur@exa...	Bireysel	2025-12-24

Figure 2. *Customer Table*

2. Employee Table

- **Definition:** Manages company personnel data and hierarchical structure.
- **Primary Key:** EmployeeID (INT, Identity 1,1)
- **Foreign Key:** SupervisorID references Employee(EmployeeID) (Self-Referencing Relationship).

- **Columns:** Name, Surname, Salary, PhoneNumber, Email, Position, Address, HiredDate.

EmployeeID	Name	Surname	Salary	PhoneNumber	Email	Position	Address	HiredDate
1	Osman	Yılmaz	9500,00	05001112233	osman.yilmaz...	Manager	İstanbul	2015-01-10
2	Banu	Aksoy	7200,00	05003334455	banu.aksoy@c...	Engineer	Ankara	2017-03-12
3	Ozan	Demir	6800,00	05001234567	ozan.demir@co...	Technician	İzmir	2018-07-22
4	Cemil	Can	7000,00	05007894561	cemil.can@co...	Analyst	Bursa	2019-02-14
5	Tuğçe	Aydın	6000,00	05008887766	tugce.aydin@c...	Support	Konya	2020-05-10
6	Sinan	Turan	7300,00	05009990012	sinan.turan@co...	Engineer	Eskişehir	2016-04-18
7	Gökhan	Yücel	6700,00	05006665544	gokhan.yucel@...	Designer	Kayseri	2018-01-25
8	Emre	Taşkın	7200,00	05007774422	emre.taskin@c...	Technician	Gaziantep	2019-11-11
9	Pelin	Kılıç	6900,00	05004561239	pelin.kilic@co...	HR Specialist	Mersin	2021-03-07
10	Hande	Güneş	7600,00	05009384756	hande.gunes@...	Supervisor	Tekirdağ	2014-12-02
11	Yiğit	Bulut	5800,00	05002568974	yigit.bulut@co...	Support	Muğla	2022-04-04
12	Serkan	Çetin	6300,00	05003847562	serkan.cetin@c...	Sales	Trabzon	2018-08-16
13	Mert	Kaptan	6400,00	05007452319	mert.kaptan@c...	HR	Balıkesir	2017-10-01
14	Furkan	Toprak	7000,00	05006123478	furkan.toprak@...	Marketing	Adana	2015-06-05
15	Gizem	Altan	6100,00	05003457812	gizem.altan@c...	Admin Assistant	Edirne	2020-09-21
16	Hasan	Şen	7500,00	05009234581	hasan.sen@co...	Engineer	Rize	2013-05-12
17	Dilek	Soylu	6800,00	05004321234	dilek.soylu@co...	Coordinator	Aydın	2019-01-29
18	Caner	Bozkurt	5400,00	05006549871	caner.bozkurt@...	Technician	Van	2021-07-15
19	Irmak	Yücel	7700,00	05007891234	irmak.yucel@c...	Supervisor	Malatya	2014-03-03
20	Barış	Öztürk	6900,00	05005678123	baris.ozturk@c...	Logistics	Kütahya	2017-11-02
21	Deniz	Akın	6500,00	05009998877	deniz.akin@co...	Designer	Sakarya	2021-02-18

Figure 3. *Employee Table*

3. Product Table

- **Definition:** Inventory of products manufactured or supplied.
- **Primary Key:** ProductID (INT, Identity 1,1)
- **Foreign Keys:**
  - SupplierID references Supplier(SupplierID)
  - MachineID references Machine(MachineID)
- **Constraints:**
  - **Default:** Stock initializes to 0.
- **Index:** IX_Product_Category on Category column for faster search performance.
- **Columns:** ProductName, UnitPrice, Cost, ManufacturedDate, QualityStatus.

ProductID	ProductName	UnitPrice	Category	Cost	Stock	Manufactured...	SupplierID	MachineID	QualityStatus
1	Metal Gövde A1	250,00	Metal Parça	180,00	120	2022-01-10	1	3	NULL
2	Elektronik Kont...	480,00	Elektronik	350,00	85	2021-03-15	2	4	NULL
3	Motor Sürücü ...	950,00	Motor	720,00	60	2020-07-22	19	5	NULL
4	Plastik Kapak P...	40,00	Plastik	15,00	300	2021-02-10	10	9	NULL
5	Hassas Sensör S...	780,00	Sensör	620,00	40	2023-04-01	23	25	NULL
6	Fiber Kablo F12	150,00	Kablo	90,00	500	2022-05-12	22	7	NULL
7	Servo Motor S...	1200,00	Motor	900,00	35	2020-11-05	12	8	NULL
8	Montaj Parçası ...	95,00	Montaj	55,00	200	2022-03-18	14	11	NULL
9	Robotik Kol Bile...	2100,00	Robotik	1650,00	25	2021-01-25	20	20	NULL
10	Kauçuk Yastık K...	65,00	Kauçuk	30,00	260	2023-02-01	5	14	NULL
11	Soğutma Ünite...	3300,00	Soğutma	2500,00	10	2020-09-21	13	12	NULL
12	Polimer Kasnak...	180,00	Polimer	70,00	140	2019-06-15	20	16	NULL
13	Hidrolik Pompa...	2700,00	Hidrolik	2100,00	18	2022-08-11	16	17	NULL
14	Montaj Aparatı ...	120,00	Montaj	65,00	180	2021-10-02	15	15	NULL
15	Servo Kontrol Ü...	680,00	Elektronik	450,00	55	2023-03-12	7	13	NULL
16	Optik Lens OL25	550,00	Optik	320,00	22	2020-12-05	25	18	NULL
17	Endüstriyel Fan ...	1500,00	Soğutma	1100,00	15	2021-07-30	6	6	NULL
18	Konveyör Zincir...	230,00	Konveyör	130,00	95	2022-09-01	7	2	NULL
19	Elektrik Bağlant...	55,00	Kablo	20,00	400	2023-01-09	4	10	NULL
20	Motor Vidası M...	12,00	Vida	2,00	800	2021-06-22	1	1	NULL
21	Boru Seti BS40	340,00	Metal Parça	220,00	140	2019-11-14	11	24	NULL
22	Plastik Panel PP...	90,00	Plastik	30,00	160	2020-08-17	10	22	NULL

Figure 4. *Product Table*

4. Machine Table

- **Definition:** Tracks machinery used in production and their maintenance status.
- **Primary Key:** MachineID (INT, Identity 1,1)
- **Constraints:**
  - **Check Constraint:** Status must be one of: 'Active', 'UnderMaintenance', 'Inactive'.
  - **Default:** Status defaults to 'Active'.
- **Columns:** MachineName, PurchaseDate, Location, LastMaintenance, NextMaintenance, Capacity.

MachineID	MachineName	PurchaseDate	Status	Location	LastMaintenan...	NextMaintena...	Capacity
1	CNC Torna 2000	2015-01-10	Active	Atölye 1	2023-01-15	2023-07-15	120
2	Endüstriyel Kesi...	2016-03-12	Active	Atölye 2	2023-02-01	2023-08-01	95
3	Hidrolik Pres 300	2017-07-22	UnderMaintena...	Bakım Alanı	2022-12-05	2023-02-15	150
4	Lazer Kesim L-5...	2019-02-14	Active	Kesim Bölümü	2023-03-10	2023-09-10	80
5	Kaynak Robotu ...	2020-05-10	Active	Montaj Hattı A	2023-04-01	2023-10-01	60
6	Montaj Kolabor...	2018-11-11	Active	Montaj Hattı B	2023-03-18	2023-09-18	50
7	Konveyör Siste...	2014-09-21	Inactive	Depo 1	2022-11-02	2023-06-02	200
8	CNC Freze 1500	2013-06-05	Active	Atölye 1	2023-01-03	2023-07-03	140
9	Bükme Makine...	2016-12-02	Active	Atölye 3	2023-02-22	2023-08-22	110
10	Paketleme Rob...	2017-04-18	Active	Paketleme Alanı	2023-04-12	2023-10-12	70
11	Kompresör K-45	2015-10-29	UnderMaintena...	Bakım Alanı	2023-01-20	2023-05-20	85
12	Soğutma Ünite...	2016-08-01	Active	Soğutma Odası	2023-03-01	2023-09-01	130
13	Otomatik Vidal...	2019-03-07	Active	Montaj Hattı C	2023-02-15	2023-08-15	55
14	Darbeli Pres DP...	2014-05-12	Inactive	Depo 2	2022-10-01	2023-04-01	160
15	Endüstriyel Fırı...	2018-01-29	Active	Isıtma Alanı	2023-03-18	2023-09-18	200
16	Test Cihazı T-20	2020-07-15	Active	Test Lab	2023-05-10	2023-11-10	30
17	Talaşlı İmalat M...	2015-11-11	Active	Atölye 2	2023-04-22	2023-10-22	90
18	CNC Router R700	2017-02-18	Active	Ahşap Üretim	2023-02-11	2023-08-11	75
19	Endüstriyel Karı...	2016-04-30	Active	Kimya Bölümü	2023-01-19	2023-07-19	45
20	Otomatik Etiket...	2019-06-12	Active	Paketleme Alanı	2023-03-10	2023-09-10	65
21	Montaj Press A...	2018-12-12	Active	Montaj Hattı D	2023-02-02	2023-08-02	100
22	Robotik Kol RX-...	2020-09-09	Active	Robotik Hat	2023-04-15	2023-10-15	50

Figure 5. *Machine Table*

5. Invoice Table

- **Definition:** Stores official billing records for customers.
- **Primary Key:** InvoiceID (INT, Identity 1,1)
- **Foreign Keys:**
  - CustomerID references Customer(CustomerID)
  - ProductID references Product(ProductID)
- **Computed Column:** TotalWithTax is calculated automatically as TotalAmount * (1 + TaxRate / 100.0).
- **Constraints:**
  - **Default:** TaxRate defaults to 20.0.
- **Columns:** InvoiceDate, TotalAmount, TaxOfficeNumber, InvoiceType, InvoicingParty.

InvoiceID	InvoiceDate	CustomerID	ProductID	TotalAmount	TaxRate	TaxOfficeNum...	InvoiceType	InvoicingParty	InvoicedParty	TotalWithTax
1	2023-01-05	1	1	850,00	20,00	İST-KDK-01	Sales	Dene Teknoloji	Ali Yılmaz	1020,000000000
2	2023-01-10	2	2	1200,00	18,00	ANK-MLT-02	Sales	Dene Teknoloji	Ayşe Demir	1416,000000000
3	2023-01-14	3	3	450,00	20,00	İZM-KRS-11	Return	Mehmet Kaya	Dene Teknoloji	540,000000000
4	2023-01-18	4	4	980,00	20,00	BUR-TRM-09	Sales	Dene Teknoloji	Elif Koç	1176,000000000
5	2023-01-22	5	5	1500,00	18,00	ANT-GLM-03	Sales	Dene Teknoloji	Canan Aydın	1770,000000000
6	2023-02-01	6	6	620,00	20,00	KON-YNS-07	Sales	Dene Teknoloji	Burak Şahin	744,000000000
7	2023-02-04	7	7	2200,00	20,00	ESK-FRM-02	Sales	Dene Teknoloji	Zeynep Arslan	2640,000000000
8	2023-02-09	8	8	980,00	18,00	KYS-DMR-10	Sales	Dene Teknoloji	Fatma Çelik	1156,400000000
9	2023-02-12	9	9	3600,00	20,00	GZT-TBR-21	Sales	Dene Teknoloji	Ahmet Öz	4320,000000000
10	2023-02-18	10	10	740,00	20,00	MRS-ERL-33	Return	Selin Er	Dene Teknoloji	888,000000000
11	2023-03-03	11	11	650,00	20,00	TKR-KPL-44	Sales	Dene Teknoloji	Hülya Kaplan	780,000000000
12	2023-03-07	12	12	4300,00	18,00	MUG-BLT-22	Sales	Dene Teknoloji	Orhan Bulut	5074,000000000
13	2023-03-09	13	13	1200,00	20,00	TRB-DNZ-12	Sales	Dene Teknoloji	Deniz Taş	1440,000000000
14	2023-03-15	14	14	1700,00	20,00	BLK-KAR-56	Sales	Dene Teknoloji	Gamze Kar	2040,000000000
15	2023-03-20	15	15	2500,00	18,00	ADN-UZN-78	Sales	Dene Teknoloji	Murat Uzun	2950,000000000
16	2023-04-02	16	16	880,00	20,00	EDR-TNC-03	Return	Aslı Tunç	Dene Teknoloji	1056,000000000
17	2023-04-06	17	17	1980,00	20,00	RİZ-GOK-19	Sales	Dene Teknoloji	Kerem Gök	2376,000000000
18	2023-04-10	18	18	640,00	20,00	AYD-KIR-88	Sales	Dene Teknoloji	Nazlı Kır	768,000000000
19	2023-04-15	19	19	1100,00	20,00	VAN-ATS-04	Sales	Dene Teknoloji	Uğur Ateş	1320,000000000
20	2023-04-19	20	20	2850,00	20,00	MLT-BRN-90	Sales	Dene Teknoloji	Sevgi Baran	3420,000000000
21	2023-05-03	21	21	720,00	20,00	KUT-CNK-11	Return	Tolga Cenk	Dene Teknoloji	864,000000000
22	2023-05-08	22	22	450,00	20,00	SAK-GUR-14	Sales	Dene Teknoloji	Melis Gür	540,000000000

Figure 6. *Invoice Table*

6. Payment Table

- **Definition:** Records financial transactions linked to invoices.
- **Primary Key:** PaymentID (INT, Identity 1,1)
- **Foreign Key:** InvoiceID references Invoice(InvoiceID)
- **Constraints:**
  - **Check Constraint:** PaymentAmount must be greater than 0.
- **Index:** IX_Payment_Date on PaymentDate column.
- **Columns:** PaymentType, PaymentDate, SwiftNo, PaymentMethod, ApprovedBy, Payee.

PaymentID	InvoiceID	PaymentType	PaymentDate	PaymentAmo...	SwiftNo	PaymentMeth...	ApprovedBy	Payee
1	1	Sale Payment	2023-01-06	850,00	TR001XYZ001	EFT	Ahmet Kar	Dene Teknoloji
2	2	Sale Payment	2023-01-12	1200,00	TR002XYZ002	EFT	Elif Kaya	Dene Teknoloji
3	3	Return Refund	2023-01-16	450,00	*NULL*	CASH	Burak Demir	Mehmet Kaya
4	4	Sale Payment	2023-01-19	980,00	TR004XYZ004	CREDIT	Selin Ak	Dene Teknoloji
5	5	Sale Payment	2023-01-25	1500,00	TR005XYZ005	EFT	Merve Çınar	Dene Teknoloji
6	6	Sale Payment	2023-02-03	620,00	*NULL*	CASH	Zeynep Kurt	Dene Teknoloji
7	7	Sale Payment	2023-02-07	2200,00	TR007XYZ007	EFT	Efe Tunç	Dene Teknoloji
8	8	Sale Payment	2023-02-11	980,00	*NULL*	CASH	Savaş Yalçın	Dene Teknoloji
9	9	Sale Payment	2023-02-14	3600,00	TR009XYZ009	EFT	Aslı Tek	Dene Teknoloji
10	10	Return Refund	2023-02-20	740,00	*NULL*	CASH	Pelin Ada	Selin Er
11	11	Sale Payment	2023-03-04	650,00	*NULL*	CASH	Koray Alp	Dene Teknoloji
12	12	Sale Payment	2023-03-10	4300,00	TR012XYZ012	EFT	Yusuf Ateş	Dene Teknoloji
13	13	Sale Payment	2023-03-12	1200,00	*NULL*	CREDIT	Murat Yılmaz	Dene Teknoloji
14	14	Sale Payment	2023-03-17	1700,00	TR014XYZ014	EFT	Gültekin Ar	Dene Teknoloji
15	15	Sale Payment	2023-03-21	2500,00	TR015XYZ015	EFT	Melis Tuncer	Dene Teknoloji
16	16	Return Refund	2023-04-05	880,00	*NULL*	CASH	Serkan Acar	Aslı Tunç
17	17	Sale Payment	2023-04-09	1980,00	TR017XYZ017	EFT	Filiz Er	Dene Teknoloji
18	18	Sale Payment	2023-04-12	640,00	*NULL*	CASH	Hande Solmaz	Dene Teknoloji
19	19	Sale Payment	2023-04-17	1100,00	*NULL*	CREDIT	Mert Can	Dene Teknoloji
20	20	Sale Payment	2023-04-22	2850,00	TR020XYZ020	EFT	Berk Yıldız	Dene Teknoloji
21	21	Return Refund	2023-05-04	720,00	*NULL*	CASH	Nesrin Ak	Tolga Cenk
22	22	Sale Payment	2023-05-09	450,00	*NULL*	CASH	Ayhan Yurt	Dene Teknoloji

Figure 7. *Payment Table*

7. QualityCheck Table

- **Definition:** Stores inspection results for manufactured products.
- **Primary Key:** QualityCheckID (INT, Identity 1,1)
- **Foreign Keys:**
  - ProductID references Product(ProductID)
  - CheckedByEmployeeID references Employee(EmployeeID)
- **Constraints:**
  - **Check Constraint:** Result must be either 'Pass' or 'Fail'.
- **Columns:** MachineID, CheckDate.

QualityCheckID	ProductID	MachineID	CheckDate	Result	CheckedByEm...
1	1	1	2023-01-10	Pass	1
2	2	3	2023-01-12	Pass	2
3	3	4	2023-01-15	Fail	3
4	4	2	2023-01-18	Pass	4
5	5	5	2023-01-20	Pass	5
6	6	6	2023-02-01	Pass	6
7	7	7	2023-02-05	Fail	7
8	8	8	2023-02-08	Pass	8
9	9	9	2023-02-10	Pass	9
10	10	10	2023-02-14	Fail	10
11	11	11	2023-03-01	Pass	11
12	12	12	2023-03-03	Pass	12
13	13	13	2023-03-05	Fail	13
14	14	14	2023-03-10	Pass	14
15	15	15	2023-03-12	Pass	15
16	16	16	2023-04-01	Pass	16
17	17	17	2023-04-04	Fail	17
18	18	18	2023-04-06	Pass	18
19	19	19	2023-04-09	Pass	19
20	20	20	2023-04-15	Fail	20
21	21	21	2023-05-01	Pass	21
22	22	22	2023-05-03	Pass	22

Figure 8. *QualityCheck Table*

8. Supplier Table

- **Definition:** Manages external vendors supplying raw materials or parts.
- **Primary Key:** SupplierID (INT, Identity 1,1)
- **Columns:** SupplierName, Address, Email, PhoneNumber, ContactPerson, Country, Rating, ReceivedProduct.

SupplierID	SupplierName	Address	Email	PhoneNumber	ContactPerson	Country	Rating	PartnerShip	ReceivedProd...
1	Akçelik Tedarik	İstanbul	info@akcelik.c...	02125551212	Murat Aydın	Türkiye	5	2015-01-10	Metal Parça
2	MegaTech Com...	Ankara	support@mega...	03124567890	Elif Aksoy	Türkiye	4	2016-03-12	Elektronik Kart
3	Efe Makina	İzmir	iletisim@efema...	02324561234	Efe Kumru	Türkiye	5	2014-07-22	Motor Aksamı
4	Kuzey Parça	Bursa	info@kuzeypar...	02245781234	Zeynep Çelik	Türkiye	3	2018-02-14	Vida & Civata
5	Doğu Endüstri	Gaziantep	destek@dogue...	03424569870	Hakan Öztürk	Türkiye	4	2017-05-10	Kauçuk Parça
6	Global Parts Co	Berlin	contact@globa...	004930112233	Anna Müller	Almanya	5	2013-11-11	Hassas Bileşen
7	TechLine GmbH	Frankfurt	info@techline.de	004969445566	Lukas Schneider	Almanya	4	2018-08-01	Kontrol Paneli
8	EastAsia Supply	Seul	support@easta...	008223154400	Kim Jisoo	Kore	5	2019-04-18	Elektrik Kablosu
9	KabloMaster	Konya	iletisim@kablos...	03324447766	Serkan Kar	Türkiye	3	2016-09-21	Kablo Takımı
10	BolTech	Kayseri	info@boltech.c...	03526789900	Fatih Bora	Türkiye	4	2014-06-05	Plastik Kapak
11	Nordic Steel AB	Stockholm	support@nordi...	004670998877	Erik Larsson	İsveç	5	2015-12-02	Boru Seti
12	EuroMach Ltd	Londra	contact@euro...	004420778899	James Smith	İngiltere	4	2017-03-07	Servo Motor
13	SinoTech Indust...	Şanghay	info@sinotech....	008621334455	Li Wei	Çin	5	2012-05-12	Soğutma Ünitesi
14	Mediterranean ...	İzmir	iletisim@medp...	02324321234	Seda Karaca	Türkiye	4	2018-01-29	Montaj Parçası
15	Anadolu Elektrik	Adana	destek@anadol...	03224561200	Levent Sönmez	Türkiye	3	2020-07-15	Kablo Demeti
16	Hitech Nordic	Oslo	info@hitechnor...	004722665544	Ola Hansen	Norveç	5	2013-10-01	Hidrolik Pompa
17	MegaSteel	Kocaeli	info@megastee...	02624567890	Ayhan Erel	Türkiye	4	2015-01-25	Çelik Parça
18	PowerDrive	İstanbul	support@powe...	02125559877	Gökhan Uçar	Türkiye	5	2016-11-02	Motor Sürücü
19	Oceanic Plastics	İzmir	destek@ocean...	02324412345	Deniz Ok	Türkiye	3	2018-04-18	Polimer Parça
20	Future Compo...	Tokyo	info@futureco...	008133556677	Sato Hiroshi	Japonya	5	2014-03-03	Robotik Parça
21	MetalTech	Bursa	info@metaltec...	02246789912	Yasin Ar	Türkiye	4	2017-11-11	Metal Gövde
22	FiberLux	İstanbul	contact@fiberl...	02126663344	Burcu Karel	Türkiye	3	2019-12-12	Fiber Kablo

Figure 9. *Supplier Table*

9. Partner Table

- **Definition:** Stores information about business partners in collaborative projects.
- **Primary Key:** PartnerID (INT, Identity 1,1)
- **Columns:** PartnerName, PartnerType, Country, PartnershipDate, Email, PhoneNumber.

PartnerID	PartnerName	PartnerType	Country	PartnershipDate	Email	PhoneNumber
1	Alpha Solutions	Technology	Türkiye	2018-01-10	info@alphasolu...	02121234567
2	Beta Logistics	Logistics	Türkiye	2017-03-12	support@betal...	02125553377
3	Orion Global	Manufacturing	Almanya	2016-07-22	contact@orion...	004930112233
4	NovaTech	R&D	İsveç	2019-02-14	info@novatech...	004670998877
5	Sunrise Import	Import	Çin	2015-05-10	sales@sunrise.cn	008621334455
6	BlueOcean Trade	Export	İngiltere	2020-11-11	info@blueocea...	004420778899
7	GreenEnergy Pa...	Energy	Norveç	2018-08-01	contact@green...	004722665544
8	TechBridge	Technology	ABD	2014-09-21	info@techbridg...	001415998877
9	Horizon Machi...	Manufacturing	Japonya	2016-06-05	support@horiz...	008133556677
10	EuroSupply	Logistics	Hollanda	2017-12-02	office@eurosu...	003120778855
11	AsiaTrade Co	Import	Singapur	2015-04-18	info@asiatrade....	006534128800
12	GlobalVision	Consulting	Fransa	2019-01-29	support@globa...	0033144556677
13	PrimeTech	Technology	İspanya	2020-07-15	info@primetec...	003491223344
14	HyperLink	Software	Türkiye	2013-10-01	info@hyperlink...	02124567890
15	OptiSystems	Optics	İtalya	2016-01-25	sales@optisyste...	003906558877
16	ProConnect	Logistics	Türkiye	2018-11-02	info@proconne...	02124446655
17	RedStar Robotics	Manufacturing	Japonya	2014-04-18	contact@redsta...	008133667788
18	SkyTech	Technology	Kore	2019-03-03	info@skytech.kr	008223154400
19	FutureWorks	R&D	ABD	2015-11-11	team@futurew...	001305778899
20	GoldenGate	Consulting	Türkiye	2021-02-18	info@goldenga...	02123334455
21	BrightLine	Marketing	Kanada	2017-04-30	contact@bright...	001416778899
22	DeepSea Trade	Export	Norveç	2016-06-12	info@deepsea.no	004723667788

Figure 10. *Partner Table*

3.4. Views

Four views have been created in the database to facilitate reporting and simplify complex queries. These views retrieve data from multiple tables and perform calculations.

1. VW_PRODUCT_QUALITY_RESULTS

- **Definition:** Consolidates quality control results by joining data from Product, Employee, and Machine tables. It allows tracking which employee checked which product using which machine and the result.

```sql
CREATE VIEW VW_PRODUCT_QUALITY_RESULTS
AS
SELECT
    qc.QualityCheckID,
    qc.CheckDate,
    qc.Result,
    p.ProductID,
    p.ProductName,
    p.Category,
    e.EmployeeID,
    e.Name + ' ' + e.Surname AS FullName,
    e.Position,
    m.MachineID,
    m.MachineName,
    m.Status AS MachineStatus
FROM QualityCheck qc
INNER JOIN Product p
    ON qc.ProductID = p.ProductID
INNER JOIN Employee e
    ON qc.CheckedByEmployeeID = e.EmployeeID
LEFT JOIN Machine m
    ON qc.MachineID = m.MachineID;
```

Figure 11. *Codes for the view named VW_PRODUCT_QUALITY_RESULTS.*

	QualityCheckID	CheckDate	Result	ProductID	ProductName	Category	EmployeeID	FullName	Position	MachineID	MachineName	MachineStatus
1	1	2023-01-10	Pass	1	Metal Gövde A1	Metal Parça	1	Osman Yılmaz	Manager	1	CNC Toma 2000	Active
2	2	2023-01-12	Pass	2	Elektronik Kontrol Kartı K10	Elektronik	2	Banu Aksoy	Engineer	3	Hidrolik Pres 300	UnderMaintenance
3	3	2023-01-15	Fail	3	Motor Sürücü M2	Motor	3	Ozan Demir	Technician	4	Lazer Kesim L-500	Active
4	4	2023-01-18	Pass	4	Plastik Kapak P500	Plastik	4	Cemil Can	Analyst	2	Endüstriyel Kesici X5	Active
5	5	2023-01-20	Pass	5	Hassas Sensör S20	Sensör	5	Tuğçe Aydın	Support	5	Kaynak Robotu R2	Active
6	6	2023-02-01	Pass	6	Fiber Kablo F12	Kablo	6	Sinan Turan	Engineer	6	Montaj Kolaboratif ...	Active
7	7	2023-02-05	Fail	7	Servo Motor SM150	Motor	7	Gökhan Yü...	Designer	7	Konveyör Sistem C...	Inactive
8	8	2023-02-08	Pass	8	Montaj Parçası MP30	Montaj	8	Emre Taşkın	Technician	8	CNC Freze 1500	Active
9	9	2023-02-10	Pass	9	Robotik Kol Bileşeni RBX	Robotik	9	Pelin Kılıç	HR Spec...	9	Bükme Makinesi B...	Active
10	10	2023-02-14	Fail	10	Kauçuk Yastık KY-80	Kauçuk	10	Hande Güneş	Supervisor	10	Paketleme Robotu...	Active
11	11	2023-03-01	Pass	11	Soğutma Ünitesi S200	Soğutma	11	Yiğit Bulut	Support	11	Kompresör K-45	UnderMaintenance
12	12	2023-03-03	Pass	12	Polimer Kasnak PK100	Polimer	12	Serkan Çetin	Sales	12	Soğutma Ünitesi S-...	Active
13	13	2023-03-05	Fail	13	Hidrolik Pompa HP300	Hidrolik	13	Mert Kaptan	HR	13	Otomatik Vidalama...	Active
14	14	2023-03-10	Pass	14	Montaj Aparatı MA44	Montaj	14	Furkan Topr...	Marketing	14	Darbeli Pres DP-90	Inactive
15	15	2023-03-12	Pass	15	Servo Kontrol Ünitesi SK...	Elektronik	15	Gizem Altan	Admin As...	15	Endüstriyel Fırın F-...	Active
16	16	2023-04-01	Pass	16	Optik Lens OL25	Optik	16	Hasan Şen	Engineer	16	Test Cihazı T-20	Active
17	17	2023-04-04	Fail	17	Endüstriyel Fan F900	Soğutma	17	Dilek Soylu	Coordinator	17	Talaşlı İmalat Maki...	Active
18	18	2023-04-06	Pass	18	Konveyör Zinciri KZ75	Konveyör	18	Caner Bozkurt	Technician	18	CNC Router R700	Active
19	19	2023-04-09	Pass	19	Elektrik Bağlantı Kablosu ...	Kablo	19	Irmak Yücel	Supervisor	19	Endüstriyel Karıştırı...	Active
20	20	2023-04-15	Fail	20	Motor Vidası MV10	Vida	20	Barış Öztürk	Logistics	20	Otomatik Etiketlem...	Active
21	21	2023-05-01	Pass	21	Boru Seti BS40	Metal Parça	21	Deniz Akın	Designer	21	Montaj Press AP-10	Active
22	22	2023-05-03	Pass	22	Plastik Panel PP22	Plastik	22	Eren Özdemir	Technician	22	Robotik Kol RX-90	Active
23	23	2023-05-05	Fail	23	Robotik Aktüatör RA50	Robotik	23	Mine Çevik	Sales	23	Toz Boya Fırını BF-	Active

Sorgu başarıyla yürütüldü. | localhost (16.0 RTM) | DESKTOP-CUSP7CS\Ozen (53) | DENE | 00:00:00 | 25 satır

Figure 12. *The results table returned when the query SELECT * FROM VW_PRODUCT_QUALITY_RESULTS is executed.*

2. VW_PRODUCT_SUPPLIER

- **Definition:** Lists detailed product information alongside supplier details. It uses a CASE statement to indicate whether a product has a supplier ('Has Supplier') or not ('No Supplier').

```sql
CREATE VIEW VW_PRODUCT_SUPPLIER
AS
SELECT
    p.ProductID,
    p.ProductName,
    p.Category,
    p.UnitPrice,
    p.Cost,
    p.Stock,
    p.ManufacturedDate,
    s.SupplierID,
    s.SupplierName,
    s.Country,
    s.Rating AS SupplierRating,
    s.ContactPerson,
    CASE
        WHEN s.SupplierID IS NULL THEN 'No Supplier'
        ELSE 'Has Supplier'
    END AS SupplierStatus
FROM Product p
LEFT JOIN Supplier s
    ON p.SupplierID = s.SupplierID;
```

Figure 13. *Codes for the view named VW_PRODUCT_SUPPLIER.*

	ProductID	ProductName	Category	UnitPrice	Cost	Stock	ManufacturedDate	SupplierID	SupplierName	Country	SupplierRating	ContactPerson	SupplierStatus
1	1	Metal Gövde A1	Metal Parça	250.00	180.00	120	2022-01-10	1	Akçelik Tedarik	Türkiye	5	Murat Aydın	Has Supplier
2	2	Elektronik Kontrol Kartı K10	Elektronik	480.00	350.00	85	2021-03-15	2	MegaTech Components	Türkiye	4	Elif Aksoy	Has Supplier
3	3	Motor Sürücü M2	Motor	950.00	720.00	60	2020-07-22	19	Oceanic Plastics	Türkiye	3	Deniz Ok	Has Supplier
4	4	Plastik Kapak P500	Plastik	40.00	15.00	300	2021-02-10	10	BolTech	Türkiye	4	Fatih Bora	Has Supplier
5	5	Hassas Sensör S20	Sensör	780.00	620.00	40	2023-04-01	23	AsiaConnect	Çin	5	Chen Wong	Has Supplier
6	6	Fiber Kablo F12	Kablo	150.00	90.00	500	2022-05-12	22	FiberLux	Türkiye	3	Burcu Karel	Has Supplier
7	7	Servo Motor SM150	Motor	1200.00	900.00	35	2020-11-05	12	EuroMach Ltd	İngiltere	4	James Smith	Has Supplier
8	8	Montaj Parçası MP30	Montaj	95.00	55.00	200	2022-03-18	14	Mediterranean Parts	Türkiye	4	Seda Karaca	Has Supplier
9	9	Robotik Kol Bileşeni RBX	Robotik	2100.00	1650...	25	2021-01-25	20	Future Components	Japo...	5	Sato Hiroshi	Has Supplier
10	10	Kauçuk Yastık KY-80	Kauçuk	65.00	30.00	260	2023-02-01	5	Doğu Endüstri	Türkiye	4	Hakan Öztürk	Has Supplier
11	11	Soğutma Ünitesi S200	Soğutma	3300.00	2500...	10	2020-09-21	13	SinoTech Industrial	Çin	5	Li Wei	Has Supplier
12	12	Polimer Kasnak PK100	Polimer	180.00	70.00	140	2019-06-15	20	Future Components	Japo...	5	Sato Hiroshi	Has Supplier
13	13	Hidrolik Pompa HP300	Hidrolik	2700.00	2100...	18	2022-08-11	16	Hitech Nordic	Norveç	5	Ola Hansen	Has Supplier
14	14	Montaj Aparatı MA44	Montaj	120.00	65.00	180	2021-10-02	15	Anadolu Elektrik	Türkiye	3	Levent Sön...	Has Supplier
15	15	Servo Kontrol Ünitesi SK...	Elektronik	680.00	450.00	55	2023-03-12	7	TechLine GmbH	Alma...	4	Lukas Schn...	Has Supplier
16	16	Optik Lens OL25	Optik	550.00	320.00	22	2020-12-05	25	OptiParts	Türkiye	4	Semih Demir	Has Supplier
17	17	Endüstriyel Fan F900	Soğutma	1500.00	1100...	15	2021-07-30	6	Global Parts Co	Alma...	5	Anna Müller	Has Supplier
18	18	Konveyör Zinciri KZ75	Konveyör	230.00	130.00	95	2022-09-01	7	TechLine GmbH	Alma...	4	Lukas Schn...	Has Supplier
19	19	Elektrik Bağlantı Kablosu ...	Kablo	55.00	20.00	400	2023-01-09	4	Kuzey Parça	Türkiye	3	Zeynep Çelik	Has Supplier
20	20	Motor Vidası MV10	Vida	12.00	2.00	800	2021-06-22	1	Akçelik Tedarik	Türkiye	5	Murat Aydın	Has Supplier
21	21	Boru Seti BS40	Metal Parça	340.00	220.00	140	2019-11-14	11	Nordic Steel AB	İsveç	5	Erik Larsson	Has Supplier
22	22	Plastik Panel PP22	Plastik	90.00	30.00	160	2020-08-17	10	BolTech	Türkiye	4	Fatih Bora	Has Supplier

Sorgu başarıyla yürütüldü. | localhost (16.0 RTM) | DESKTOP-CUSP7CS\Ozen (54) | DENE | 00:00:00 | 25 satır

Figure 14. *The results table returned when the query SELECT * FROM VW_PRODUCT_SUPPLIER is executed.*

3. VW_CUSTOMER_INVOICE_SUMMARY

- **Definition:** Provides a financial summary for each customer using aggregation functions (COUNT, SUM, AVG, MAX). It shows the total invoice count, total debt with tax, and the dates of the first and last invoices.

```sql
CREATE VIEW VW_CUSTOMER_INVOICE_SUMMARY
AS
SELECT
    c.CustomerID,
    c.Name AS CustomerName,
    c.CustomerType,
    c.Email,
    c.PhoneNumber,
    COUNT(i.InvoiceID) AS InvoiceCount,
    SUM(i.TotalAmount) AS TotalInvoiceAmount,
    SUM(i.TotalWithTax) AS TotalInvoiceWithTax,
    AVG(i.TotalAmount) AS AverageInvoiceAmount,
    MAX(i.InvoiceDate) AS LastInvoiceDate,
    MIN(i.InvoiceDate) AS FirstInvoiceDate
FROM Customer c
LEFT JOIN Invoice i
    ON c.CustomerID = i.CustomerID
GROUP BY
    c.CustomerID,
    c.Name,
    c.CustomerType,
    c.Email,
    c.PhoneNumber;
```

Figure 15. *Codes for the view named VW_CUSTOMER_INVOICE_SUMMARY.*

	CustomerID	CustomerName	CustomerType	Email	PhoneNumber	InvoiceCount	TotalInvoiceAmount	TotalInvoiceWithTax	AverageInvoiceAmount	LastInvoiceDate	FirstInvoiceDate
1	1	Ali Yılmaz	Bireysel	ali.yilmaz@example.com	05001112233	1	850.00	1020.000000000	850.000000	2023-01-05	2023-01-05
2	2	Ayşe Demir	Kurumsal	ayse.demir@example.com	05003334455	1	1200.00	1416.000000000	1200.000000	2023-01-10	2023-01-10
3	3	Mehmet Kaya	Bireysel	mehmet.kaya@example.com	05001234567	1	450.00	540.000000000	450.000000	2023-01-14	2023-01-14
4	4	Elif Koç	Bireysel	elif.koc@example.com	05007894561	1	980.00	1176.000000000	980.000000	2023-01-18	2023-01-18
5	5	Canan Aydın	Kurumsal	canan.aydin@example.com	05009283746	1	1500.00	1770.000000000	1500.000000	2023-01-22	2023-01-22
6	6	Burak Şahin	Bireysel	burak.sahin@example.com	05008887766	1	620.00	744.000000000	620.000000	2023-02-01	2023-02-01
7	7	Zeynep Arslan	Kurumsal	zeynep.arslan@example.c...	05009990012	1	2200.00	2640.000000000	2200.000000	2023-02-04	2023-02-04
8	8	Fatma Çelik	Bireysel	fatma.celik@example.com	05006665544	1	980.00	1156.400000000	980.000000	2023-02-09	2023-02-09
9	9	Ahmet Öz	Kurumsal	ahmet.oz@example.com	05007774422	1	3600.00	4320.000000000	3600.000000	2023-02-12	2023-02-12
10	10	Selin Er	Bireysel	selin.er@example.com	05004561239	1	740.00	888.000000000	740.000000	2023-02-18	2023-02-18
11	11	Hülya Kaplan	Kurumsal	hulya.kaplan@example.com	05009384756	1	650.00	780.000000000	650.000000	2023-03-03	2023-03-03
12	12	Orhan Bulut	Bireysel	orhan.bulut@example.com	05002568974	1	4300.00	5074.000000000	4300.000000	2023-03-07	2023-03-07
13	13	Deniz Taş	Kurumsal	deniz.tas@example.com	05003847562	1	1200.00	1440.000000000	1200.000000	2023-03-09	2023-03-09
14	14	Gamze Kar	Bireysel	gamze.kar@example.com	05007452319	1	1700.00	2040.000000000	1700.000000	2023-03-15	2023-03-15
15	15	Murat Uzun	Kurumsal	murat.uzun@example.com	05006123478	1	2500.00	2950.000000000	2500.000000	2023-03-20	2023-03-20
16	16	Aslı Tunç	Bireysel	asli.tunc@example.com	05003457812	1	880.00	1056.000000000	880.000000	2023-04-02	2023-04-02
17	17	Kerem Gök	Kurumsal	kerem.gok@example.com	05009234581	1	1980.00	2376.000000000	1980.000000	2023-04-06	2023-04-06
18	18	Nazlı Kır	Kurumsal	nazli.kir@example.com	05004321234	1	640.00	768.000000000	640.000000	2023-04-10	2023-04-10
19	19	Uğur Ateş	Bireysel	ugur.ates@example.com	05006549871	1	1100.00	1320.000000000	1100.000000	2023-04-15	2023-04-15
20	20	Sevgi Baran	Kurumsal	sevgi.baran@example.com	05007891234	1	2850.00	3420.000000000	2850.000000	2023-04-19	2023-04-19
21	21	Tolga Cenk	Bireysel	tolga.cenk@example.com	05005678123	1	720.00	864.000000000	720.000000	2023-05-03	2023-05-03
22	22	Melis Gür	Bireysel	melis.gur@example.com	05009998877	1	450.00	540.000000000	450.000000	2023-05-08	2023-05-08
23	23	Okan Sert	Kurumsal	okan.sert@example.com	05003458912	1	2100.00	2478.000000000	2100.000000	2023-05-12	2023-05-12

Sorgu başarıyla yürütüldü. localhost (16.0 RTM) DESKTOP-CUSP7CS\Ozen (53) DENE 00:00:00 25 satır

Figure 16. *The results table returned when the query SELECT * FROM VW_CUSTOMER_INVOICE_SUMMARY is executed.*

4. VW_PAYMENT_STATUS

- **Definition:** Calculates the real-time payment status of each invoice. It compares the TotalWithTax amount with the total payments made (PaidAmount) to determine if the status is 'Paid', 'Unpaid', 'Partially Paid', or 'Overpaid'.

```sql
CREATE VIEW VW_PAYMENT_STATUS
AS
SELECT
    c.CustomerID,
    c.Name AS CustomerName,
    c.CustomerType,
    i.InvoiceID,
    i.InvoiceDate,
    i.TotalAmount,
    i.TaxRate,
    i.TotalWithTax,
    ISNULL(SUM(p.PaymentAmount), 0) AS PaidAmount,
    i.TotalWithTax - ISNULL(SUM(p.PaymentAmount), 0) AS RemainingAmount,
    CASE
        WHEN ISNULL(SUM(p.PaymentAmount), 0) = 0 THEN 'Unpaid'
        WHEN i.TotalWithTax - ISNULL(SUM(p.PaymentAmount), 0) = 0 THEN 'Paid'
        WHEN i.TotalWithTax - ISNULL(SUM(p.PaymentAmount), 0) > 0 THEN 'Partially Paid'
        ELSE 'Overpaid'
    END AS PaymentStatus,
    COUNT(p.PaymentID) AS PaymentCount,
    MAX(p.PaymentDate) AS LastPaymentDate
FROM Invoice i
INNER JOIN Customer c
    ON i.CustomerID = c.CustomerID
LEFT JOIN Payment p
    ON i.InvoiceID = p.InvoiceID
GROUP BY
    c.CustomerID,
    c.Name,
    c.CustomerType,
    i.InvoiceID,
    i.InvoiceDate,
    i.TotalAmount,
    i.TaxRate,
    i.TotalWithTax;
```

Figure 17&18. *Codes for the view named VW_PAYMENT_STATUS.*

	CustomerID	CustomerName	CustomerType	InvoiceID	InvoiceDate	TotalAmount	TaxRate	TotalWithTax	PaidAmount	RemainingAmount	PaymentStatus	PaymentCount	LastPaymentDate
1	1	Ali Yilmaz	Bireysel	1	2023-01-05	850.00	20.00	1020.000000000	850.00	170.00	Partially Paid	1	2023-01-06
2	2	Ayşe Demir	Kurumsal	2	2023-01-10	1200.00	18.00	1416.000000000	1200.00	216.00	Partially Paid	1	2023-01-12
3	3	Mehmet Kaya	Bireysel	3	2023-01-14	450.00	20.00	540.000000000	450.00	90.00	Partially Paid	1	2023-01-16
4	4	Elif Koç	Bireysel	4	2023-01-18	980.00	20.00	1176.000000000	980.00	196.00	Partially Paid	1	2023-01-19
5	5	Canan Aydın	Kurumsal	5	2023-01-22	1500.00	18.00	1770.000000000	1500.00	270.00	Partially Paid	1	2023-01-25
6	6	Burak Şahin	Bireysel	6	2023-02-01	620.00	20.00	744.000000000	620.00	124.00	Partially Paid	1	2023-02-03
7	7	Zeynep Arslan	Kurumsal	7	2023-02-04	2200.00	20.00	2640.000000000	2200.00	440.00	Partially Paid	1	2023-02-07
8	8	Fatma Çelik	Bireysel	8	2023-02-09	980.00	18.00	1156.400000000	980.00	176.40	Partially Paid	1	2023-02-11
9	9	Ahmet Öz	Kurumsal	9	2023-02-12	3600.00	20.00	4320.000000000	3600.00	720.00	Partially Paid	1	2023-02-14
10	10	Selin Er	Bireysel	10	2023-02-18	740.00	20.00	888.000000000	740.00	148.00	Partially Paid	1	2023-02-20
11	11	Hülya Kaplan	Kurumsal	11	2023-03-03	650.00	20.00	780.000000000	650.00	130.00	Partially Paid	1	2023-03-04
12	12	Orhan Bulut	Bireysel	12	2023-03-07	4300.00	18.00	5074.000000000	4300.00	774.00	Partially Paid	1	2023-03-10
13	13	Deniz Taş	Kurumsal	13	2023-03-09	1200.00	20.00	1440.000000000	1200.00	240.00	Partially Paid	1	2023-03-12
14	14	Gamze Kar	Bireysel	14	2023-03-15	1700.00	20.00	2040.000000000	1700.00	340.00	Partially Paid	1	2023-03-17
15	15	Murat Uzun	Kurumsal	15	2023-03-20	2500.00	18.00	2950.000000000	2500.00	450.00	Partially Paid	1	2023-03-21
16	16	Aslı Tunç	Bireysel	16	2023-04-02	880.00	20.00	1056.000000000	880.00	176.00	Partially Paid	1	2023-04-05
17	17	Kerem Gök	Kurumsal	17	2023-04-06	1980.00	20.00	2376.000000000	1980.00	396.00	Partially Paid	1	2023-04-09
18	18	Nazlı Kır	Kurumsal	18	2023-04-10	640.00	20.00	768.000000000	640.00	128.00	Partially Paid	1	2023-04-12
19	19	Uğur Ateş	Bireysel	19	2023-04-15	1100.00	20.00	1320.000000000	1100.00	220.00	Partially Paid	1	2023-04-17
20	20	Sevgi Baran	Kurumsal	20	2023-04-19	2850.00	20.00	3420.000000000	2850.00	570.00	Partially Paid	1	2023-04-22
21	21	Tolga Cenk	Bireysel	21	2023-05-03	720.00	20.00	864.000000000	720.00	144.00	Partially Paid	1	2023-05-04
22	22	Melis Gür	Bireysel	22	2023-05-08	450.00	20.00	540.000000000	450.00	90.00	Partially Paid	1	2023-05-09
23	23	Okan Sert	Kurumsal	23	2023-05-12	2100.00	18.00	2478.000000000	2100.00	378.00	Partially Paid	1	2023-05-15

Sorgu başarıyla yürütüldü. | localhost (16.0 RTM) | DESKTOP-CUSP7CS\Ozen (53) | DENE | 00:00:00 | 25 satır

Figure 19. *The results table returned when the query SELECT * FROM VW_PAYMENT_STATUS s executed.*

3.5. Stored Procedures

A total of 11 Stored Procedures were developed within the scope of the project to ensure data integrity, implement business rules at the database level, and perform complex reporting. All procedures handle errors with TRY-CATCH blocks and guarantee data consistency using TRANSACTION.

I. Data Manipulation Procedures

1. SP_ADD_CUSTOMER

- **Definition:** Registers a new customer into the system while checking for duplicate email addresses.
- **Logic:** Checks if the email already exists. If unique, inserts the customer and returns the new CustomerID via an OUTPUT parameter.

```sql
DROP PROCEDURE IF EXISTS SP_ADD_CUSTOMER;
GO

CREATE PROCEDURE SP_ADD_CUSTOMER
    @Name NVARCHAR(100),
    @Address NVARCHAR(255),
    @PhoneNumber NVARCHAR(20),
    @Email NVARCHAR(150),
    @CustomerType NVARCHAR(50),
    @NewCustomerID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Email uniqueness check
        IF EXISTS (SELECT 1 FROM Customer WHERE Email = @Email)
        BEGIN
            ROLLBACK TRANSACTION;
            RETURN -1; -- Email conflict
        END

        -- Insert customer
        INSERT INTO Customer(Name, Address, PhoneNumber, Email, CustomerType)
        VALUES(@Name, @Address, @PhoneNumber, @Email, @CustomerType);

        SET @NewCustomerID = SCOPE_IDENTITY();

            COMMIT TRANSACTION;
            RETURN 0; -- Success
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0
                ROLLBACK TRANSACTION;
            RETURN -99; -- General error
        END CATCH
    END;
```

Figure 20&21. *Source code of SP_ADD_CUSTOMER stored procedure.*

```sql
SELECT * FROM Customer WHERE Email = 'demo.user@denetech.com';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate

Figure 22. *State of Customer table before adding a new record.*

```sql
--Insert
DECLARE @NewID INT;
EXEC SP_ADD_CUSTOMER 'Demo User', 'Teknopark Istanbul', '5559998877', 'demo.user@denetech.com', 'Kurumsal', @NewID OUTPUT;


SELECT * FROM Customer WHERE Email = 'demo.user@denetech.com';
```

100 % Sorun bulunamadı Sat: 40 Krkt: 1 BŞL

Sonuçlar | İletiler

	CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
1	27	Demo User	Teknopark Istanbul	5559998877	demo.user@denetech.com	Kurumsal	2025-12-24

Figure 23. *State of Customer table after executing SP_ADD_CUSTOMER.*

2. SP_UPDATE_CUSTOMER

- **Definition:** Updates existing customer details.
- **Logic:** Ensures the new name does not conflict with another existing customer. It allows updating critical fields like Address, Phone, and Email within a transaction.

```sql
USE DENE; -- Database name corrected
GO

-- Drop the old procedure if it exists
IF OBJECT_ID('[dbo].[SP_UPDATE_CUSTOMER]', 'P') IS NOT NULL
    DROP PROCEDURE [dbo].[SP_UPDATE_CUSTOMER];
GO

-- Recreate the procedure by fixing the Identity error
CREATE PROCEDURE [dbo].[SP_UPDATE_CUSTOMER]
    @OldCustomerID INT,
    @NewCustomerID INT, -- Parameter structure is kept unchanged (to avoid C# errors)
    @Name NVARCHAR(100),
    @Address NVARCHAR(255),
    @PhoneNumber NVARCHAR(20),
    @Email NVARCHAR(150),
    @CustomerType NVARCHAR(50),
    @TotalInvoices INT,
    @TotalSpent DECIMAL(18,2)
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- 1. Name Uniqueness Check
        IF EXISTS (SELECT 1 FROM [dbo].[Customer] WHERE Name = @Name AND CustomerID <> @OldCustomerID)
            BEGIN
                ROLLBACK TRANSACTION;
                ;THROW 50002, 'A customer with this name already exists.', 1;
            END

            -- 2. Update Operation
            -- FIX IS HERE: Removed the line "CustomerID = @NewCustomerID".
            -- Identity columns cannot be updated; only other fields are updated.
            UPDATE [dbo].[Customer]
            SET Name = @Name,
                Address = @Address,
                PhoneNumber = @PhoneNumber,
                Email = @Email,
                CustomerType = @CustomerType
            WHERE CustomerID = @OldCustomerID;

            COMMIT TRANSACTION;
            SELECT 1 AS ResultCode, 'Customer updated successfully.' AS Message;
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
            THROW;
        END CATCH
    END;
    GO
```

Figure 24&25. *Source code of SP_UPDATE_CUSTOMER procedure.*

```
USE DENE;
GO


SELECT *  FROM Customer WHERE Email = 'demo.user@denetech.com';
```

100 % 1 0

Sonuçlar | İletiler

	CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
1	27	Demo User	Teknopark Istanbul	5559998877	demo.user@denetech.com	Kurumsal	2025-12-24

Figure 26. *Customer details before the update operation.*

```
DECLARE @TargetID INT = (SELECT CustomerID FROM Customer WHERE Email = 'demo.user@denetech.com');

EXEC SP_UPDATE_CUSTOMER @TargetID, @TargetID, 'Demo User', 'YENI MERKEZ OFIS', '5559998877', 'demo.user@denetech.com', 'Kurumsal', 0, 0;


SELECT * FROM Customer WHERE Email = 'demo.user@denetech.com';
```

Sat: 59 Krkt: 1 BŞL

Sonuçlar | İletiler

	ResultCode	Message
1	1	Customer updated successfully.

	CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
1	27	Demo User	YENI MERKEZ OFIS	5559998877	demo.user@denetech.com	Kurumsal	2025-12-24

Figure 27. *Customer details updated successfully using SP_UPDATE_CUSTOMER.*

Since the CustomerID is an Identity column, its specific value cannot be predicted with certainty during testing. Therefore, a variable named @TargetID is declared to dynamically retrieve the correct ID based on the unique email address ('demo.user@denetech.com'). This ensures that the SP_UPDATE_CUSTOMER procedure targets the exact record intended for the update, preventing potential errors caused by hardcoded ID values.

3. SP_DELETE_CUSTOMER

- **Definition:** Safely deletes a customer and all dependent records (Cascade Delete logic via Transaction).
- **Logic:**
  - Deletes related Invoice records.
  - Deletes related Payment records (matched by Payee name).
  - Finally deletes the Customer.
  - If any step fails, performs a ROLLBACK.

```sql
DROP PROCEDURE IF EXISTS SP_DELETE_CUSTOMER;
GO

CREATE PROCEDURE SP_DELETE_CUSTOMER
    @CustomerID INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Müşterinin adını değişkene alalım (Payee kontrolü için)
    DECLARE @TargetCustomerName NVARCHAR(200);

    SELECT @TargetCustomerName = Customer.Name
    FROM Customer
    WHERE CustomerID = @CustomerID;

    --Müşteri kontrol kısmı
    IF @TargetCustomerName IS NULL
    BEGIN
        ;THROW 50020, 'Customer not found, deletion aborted!', 1;
        RETURN
    END

    BEGIN TRY
        BEGIN TRANSACTION;

        DELETE FROM Invoice
        WHERE CustomerID = @CustomerID;

        DELETE FROM Payment
        Where Payee = @TargetCustomerName

        DELETE FROM Customer
        Where CustomerID = @CustomerID

        COMMIT TRANSACTION;

        SELECT 1 AS ResultCode, 'Customer and all associated records deleted successfully.' AS Message;


    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        ;THROW;
    END CATCH

END;
GO
```

Figure 28&29. *Source code of SP_DELETE_CUSTOMER procedure.*

```
50
51  SELECT * FROM Customer WHERE Email = 'demo.user@denetech.com';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate
1	27	Demo User	YENI MERKEZ OFIS	5559998877	demo.user@denetech.com	Kurumsal	2025-12-24

Figure 30. *Customer records existing in database before deletion.*

```
54  DECLARE @DelID INT = (SELECT CustomerID FROM Customer WHERE Email = 'demo.user@denetech.com');
55  EXEC SP_DELETE_CUSTOMER @DelID;
56
57
58  SELECT * FROM Customer WHERE Email = 'demo.user@denetech.com';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	ResultCode	Message
1	1	Customer and all associated records deleted succ...

CustomerID	Name	Address	PhoneNumber	Email	CustomerType	RegistrationDate

Figure 31. *Confirmation that the customer record has been deleted.*

4. SP_ADD_PRODUCT

- **Definition:** Adds a new product to the inventory.
- **Logic:**
  - Validates that UnitPrice and Cost are positive.
  - Enforces that every product must be assigned to a **Machine** (Mandatory Rule).
  - Validates the SupplierID if provided.

```sql
DROP PROCEDURE IF EXISTS SP_ADD_PRODUCT;
GO --check

CREATE PROCEDURE SP_ADD_PRODUCT
    @ProductName NVARCHAR(100),
    @UnitPrice DECIMAL(12,2),
    @Category NVARCHAR(50),
    @Cost DECIMAL(12,2),
    @Stock INT = 0,
    @SupplierID INT = NULL,
    @MachineID INT -- MANDATORY (cannot be NULL)
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Price and cost check
        IF @UnitPrice <= 0 OR @Cost <= 0
        BEGIN
            ;THROW 50001, 'Price and cost must be greater than 0.', 1;
        END

        -- MachineID must be mandatory and valid
        IF @MachineID IS NULL
        BEGIN
            ;THROW 50002, 'Machine ID is mandatory for every product.', 1;
        END

        IF NOT EXISTS (SELECT 1 FROM Machine WHERE MachineID = @MachineID)
        BEGIN
            ;THROW 50003, 'Invalid machine ID.', 1;
        END

        -- SupplierID check (if provided)
        IF @SupplierID IS NOT NULL AND NOT EXISTS (SELECT 1 FROM Supplier WHERE SupplierID = @SupplierID)
        BEGIN
            ;THROW 50004, 'Invalid supplier ID.', 1;
        END

        -- Insert product
        INSERT INTO Product(ProductName, UnitPrice, Category, Cost, Stock, SupplierID, MachineID, ManufacturedDate)
        VALUES(@ProductName, @UnitPrice, @Category, @Cost, @Stock, @SupplierID, @MachineID, GETDATE());

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

Figure 32&33. *Source code of SP_ADD_PRODUCT procedure.*

```sql
SELECT * FROM Product WHERE ProductName = 'Test Urunu X';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

ProductID	ProductName	UnitPrice	Category	Cost	Stock	ManufacturedDate	SupplierID	MachineID	QualityStatus

Figure 34. *Product table query result showing no record for the new item.*

```
55  SELECT * FROM Product WHERE ProductName = 'Test Urunu X';
56
57  -- MachineID=1
58  EXEC SP_ADD_PRODUCT 'Test Urunu X', 100, 'Test Kategori', 50, 0, NULL, 1;
59
60  -- Control
61  SELECT * FROM Product WHERE ProductName = 'Test Urunu X';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	ProductID	ProductName	UnitPrice	Category	Cost	Stock	ManufacturedDate	SupplierID	MachineID	QualityStatus
1	26	Test Urunu X	100.00	Test Kategori	50.00	0	2025-12-24	NULL	1	NULL

Figure 35. *Product table displaying the newly inserted item via SP_ADD_PRODUCT.*

5. SP_UPDATE_PRODUCT_STOCK

- **Definition:** Updates the stock quantity of a specific product.
- **Logic:** Prevents stock from being set to a negative value by throwing a custom error (Error 50002).

```sql
-- ==================================================
-- SP_UPDATE_PRODUCT_STOCK Stored Procedure
-- Purpose: Update product stock quantity
-- ==================================================

USE DENE;
GO

-- Drop if exists
IF OBJECT_ID('dbo.SP_UPDATE_PRODUCT_STOCK', 'P') IS NOT NULL
    DROP PROCEDURE dbo.SP_UPDATE_PRODUCT_STOCK;
GO

-- Create stored procedure
CREATE PROCEDURE dbo.SP_UPDATE_PRODUCT_STOCK
    @ProductID INT,
    @NewStock INT
AS
BEGIN
    SET NOCOUNT ON;

    -- Validate product exists
    IF NOT EXISTS (SELECT 1 FROM dbo.Product WHERE ProductID = @ProductID)
    BEGIN
        ;THROW 50001, 'Product not found.', 1;
        RETURN;
    END

    -- Validate stock is not negative
    IF @NewStock < 0
    BEGIN
        ;THROW 50002, 'Stock cannot be negative.', 1;
        RETURN;
    END

    -- Update product stock
    UPDATE dbo.Product
    SET Stock = @NewStock
    WHERE ProductID = @ProductID;

    PRINT 'Product stock updated successfully.';
END;
GO
```

Figure 36&37. *Source code of SP_UPDATE_PRODUCT_STOCK procedure.*

```
SELECT ProductName FROM Product WHERE ProductName = 'Test Urunu X';
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	ProductName
1	Test Urunu X

Figure 38. *Current stock level before update.*

```
DECLARE @ProdID INT = (SELECT ProductID FROM Product WHERE ProductName = 'Test Urunu X');
EXEC SP_UPDATE_PRODUCT_STOCK @ProdID, 500;

-- Control
SELECT ProductName, Stock FROM Product WHERE ProductName = 'Test Urunu X';
```

00 % Sorun bulunamadı

Sonuçlar | İletiler

	ProductName	Stock
1	Test Urunu X	500

Figure 39. *Updated stock level reflecting the changes made by the stored procedure.*

6. SP_CREATE_INVOICE

- **Definition:** Generates a formal invoice for a sales transaction.
- **Logic:**
  - Enforces the rule: **"Each invoice must be related to exactly one product"**.
  - Validates Customer and Product existence.
  - Ensures the total amount is positive.

```sql
DROP PROCEDURE IF EXISTS SP_CREATE_INVOICE;
GO --Check

CREATE PROCEDURE SP_CREATE_INVOICE
    @CustomerID INT,
    @ProductID INT,
    @TotalAmount DECIMAL(12,2),
    @TaxRate DECIMAL(5,2) = 20.0,
    @TaxOfficeNumber NVARCHAR(50) = NULL,
    @InvoiceType NVARCHAR(30) = NULL,
    @InvoicingParty NVARCHAR(150),
    @InvoicedParty NVARCHAR(150),
    @NewInvoiceID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Customer check
        IF NOT EXISTS (SELECT 1 FROM Customer WHERE CustomerID = @CustomerID)
        BEGIN
            ;THROW 50015, 'Customer not found.', 1;
        END

        -- Product check (MANDATORY)
        -- Fatura Kuralı: Her fatura tam olarak bir ürünle ilişkili olmalıdır.
        IF @ProductID IS NULL OR NOT EXISTS (SELECT 1 FROM Product WHERE ProductID = @ProductID)
```

```sql
        -- Product check (MANDATORY)
        -- Fatura Kuralı: Her fatura tam olarak bir ürünle ilişkili olmalıdır.
        IF @ProductID IS NULL OR NOT EXISTS (SELECT 1 FROM Product WHERE ProductID = @ProductID)
        BEGIN
            ;THROW 50017, 'Product ID is mandatory and must be valid.', 1;
        END

        -- Amount check
        IF @TotalAmount <= 0
        BEGIN
            ;THROW 50018, 'Invoice amount must be greater than 0.', 1;
        END

        -- Create invoice
        INSERT INTO Invoice(InvoiceDate, CustomerID, ProductID, TotalAmount, TaxRate, TaxOfficeNumber, InvoiceType, InvoicingParty, InvoicedParty)
        VALUES(GETDATE(), @CustomerID, @ProductID, @TotalAmount, @TaxRate, @TaxOfficeNumber, @InvoiceType, @InvoicingParty, @InvoicedParty);

        SET @NewInvoiceID = SCOPE_IDENTITY();

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

Figure 40&41. *Source code of SP_CREATE_INVOICE procedure.*

```sql
SELECT COUNT(*) as TotalInvoices FROM Invoice;
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	TotalInvoices
1	25

Figure 42. *Total number of invoices before creating a new one.*

```sql
DECLARE @NewInv INT;
-- We invoice for the customer with ID 1 and the product with ID 1
EXEC SP_CREATE_INVOICE 1, 1, 5000, 20, 'VD-Test', 'Sales', 'Dene', 'Customer', @NewInv OUTPUT;

-- Show last invoice
SELECT TOP 1 * FROM Invoice ORDER BY InvoiceID DESC;
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	InvoiceID	InvoiceDate	CustomerID	ProductID	TotalAmount	TaxRate	TaxOfficeNumber	InvoiceType	InvoicingParty	InvoicedParty	TotalWithTax
1	26	2025-12-24	1	1	5000.00	20.00	VD-Test	Sales	Dene	Customer	6000.000000000

Figure 43. *Details of the newly created invoice record.*

II. Business Logic Procedures

7. SP_PROCESS_QUALITY_CHECK

- **Definition:** Records a quality control test and automatically updates the product's quality status.
- **Logic:**
  - Validates Product, Machine, and Employee existence.
  - Inserts a record into the QualityCheck table.
  - **Trigger-like Logic:** Updates the QualityStatus column in the Product table to match the test result ('Pass'/'Fail').

```sql
DROP PROCEDURE IF EXISTS SP_PROCESS_QUALITY_CHECK;
GO --Check

CREATE PROCEDURE SP_PROCESS_QUALITY_CHECK
    @ProductID INT,
    @MachineID INT,
    @Result NVARCHAR(10),       -- Must be 'Pass' or 'Fail'
    @CheckedByEmployeeID INT,
    @NewQualityCheckID INT OUTPUT
AS
BEGIN
    SET NOCOUNT ON;

 -- Mandatory field control
    IF @ProductID IS NULL OR @MachineID IS NULL OR @Result IS NULL OR @CheckedByEmployeeID IS NULL
    BEGIN
        THROW 50020, 'All parameters (ProductID, MachineID, Result, CheckedByEmployeeID) are mandatory.', 1;
    END

-- Result format control
    IF @Result NOT IN ('Pass', 'Fail')
    BEGIN
        THROW 50021, 'Result must be either ''Pass'' or ''Fail''.', 1;
    END

    BEGIN TRY
        BEGIN TRANSACTION;

       -- Asset control
```

```sql
        -- Asset control
        IF NOT EXISTS (SELECT 1 FROM Product WHERE ProductID = @ProductID)
        BEGIN
            THROW 50022, 'Invalid ProductID. Product not found.', 1;
        END

        IF NOT EXISTS (SELECT 1 FROM Machine WHERE MachineID = @MachineID)
        BEGIN
            THROW 50023, 'Invalid MachineID. Machine not found.', 1;
        END

        IF NOT EXISTS (SELECT 1 FROM Employee WHERE EmployeeID = @CheckedByEmployeeID)
        BEGIN
            THROW 50024, 'Invalid EmployeeID. Employee not found.', 1;
        END

       -- 1. Create QualityCheck record
        INSERT INTO QualityCheck (ProductID, MachineID, CheckDate, Result, CheckedByEmployeeID)
        VALUES (@ProductID, @MachineID, GETDATE(), @Result, @CheckedByEmployeeID);

        SET @NewQualityCheckID = SCOPE_IDENTITY();

       -- 2. Update the product's QualityStatus
        UPDATE Product
        SET QualityStatus = @Result
        WHERE ProductID = @ProductID;

```

```sql
            COMMIT TRANSACTION;
        END TRY
        BEGIN CATCH
            IF @@TRANCOUNT > 0
                ROLLBACK TRANSACTION;
            THROW;
        END CATCH
    END;
    GO
```

Figure 44&45&46. *Source code of SP_PROCESS_QUALITY_CHECK procedure.*

```
SELECT ProductID, QualityStatus FROM Product WHERE ProductID = 1;
```

	ProductID	QualityStatus
1	1	NULL

Figure 47. *Product quality status before inspection.*

```
DECLARE @QCID INT;
-- ProductID: 1, MachineID: 1, Result: Pass, EmployeeID: 1
EXEC SP_PROCESS_QUALITY_CHECK 1, 1, 'Pass', 1, @QCID OUTPUT;

-- Control
SELECT ProductID, QualityStatus FROM Product WHERE ProductID = 1;
```

	ProductID	QualityStatus
1	1	Pass

Figure 48. *Updated product status showing 'Pass' after quality check execution.*

8. SP_UPDATE_MACHINE_STATUS

- **Definition:** Updates the operational status of a machine and manages maintenance schedules.
- **Logic:** If the status is set to 'UnderMaintenance', it automatically updates the LastMaintenance date to today and calculates the NextMaintenance date (adding 3 months).

```sql
DROP PROCEDURE IF EXISTS SP_UPDATE_MACHINE_STATUS;
GO --Check

CREATE PROCEDURE SP_UPDATE_MACHINE_STATUS
    @MachineID INT,
    @Status NVARCHAR(30),
    @UpdateMaintenanceDate BIT = 0
AS
BEGIN
    SET NOCOUNT ON;

    DECLARE @ForceMaintenanceUpdate BIT;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Machine existence check
        IF NOT EXISTS (SELECT 1 FROM Machine WHERE MachineID = @MachineID)
        BEGIN
            ;THROW 50005, 'Machine not found.', 1;
        END

        -- Status validity check
        IF @Status NOT IN ('Active', 'UnderMaintenance', 'Inactive')
        BEGIN
            ;   THROW 50006, 'Invalid status. (Must be Active, UnderMaintenance, or Inactive)', 1;
        END

        -- Force maintenance date update when status is 'UnderMaintenance'
        SET @ForceMaintenanceUpdate = CASE WHEN @Status = 'UnderMaintenance' THEN 1 ELSE @UpdateMaintenanceDate END;

        -- Update machine status
        UPDATE Machine
        SET Status = @Status,
            LastMaintenance = CASE WHEN @ForceMaintenanceUpdate = 1 THEN GETDATE() ELSE LastMaintenance END,
            NextMaintenance = CASE WHEN @ForceMaintenanceUpdate = 1 THEN DATEADD(MONTH, 3, GETDATE()) ELSE NextMaintenance END
        WHERE MachineID = @MachineID;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

Figure 49&50. *Source code of SP_UPDATE_MACHINE_STATUS procedure.*

```sql
USE DENE;
GO
SELECT MachineID, Status, LastMaintenance FROM Machine WHERE MachineID = 1;
```

100 % 1 0

Sonuçlar | İletiler

	MachineID	Status	LastMaintenance
1	1	Active	2023-01-15

Figure 51. *Machine status before maintenance update.*

```
55
56  EXEC SP_UPDATE_MACHINE_STATUS 1, 'UnderMaintenance', 0;
57
58  SELECT MachineID, Status, LastMaintenance FROM Machine WHERE MachineID = 1;
```

100 % 1 0

Sonuçlar | İletiler

	MachineID	Status	LastMaintenance
1	1	UnderMaintenance	2025-12-24

Figure 52. *Machine status changed to 'Inactive' via stored procedure.*

9. SP_RECORD_QUALITYCHECK

- **Definition:** Similar to SP_PROCESS_QUALITY_CHECK but with different specific error codes for detailed debugging.
- **Logic:** Enforces specific error codes (50007-50010) for Product, Employee, and Machine validation failures.

```sql
DROP PROCEDURE IF EXISTS SP_RECORD_QUALITYCHECK;
GO --Check

CREATE PROCEDURE SP_RECORD_QUALITYCHECK
    @ProductID INT,
    @MachineID INT = NULL,
    @Result NVARCHAR(10),
    @CheckedByEmployeeID INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        BEGIN TRANSACTION;

        -- Product check
        IF NOT EXISTS (SELECT 1 FROM Product WHERE ProductID = @ProductID)
        BEGIN
            ;THROW 50007, 'Product not found.', 1;
        END

        -- Employee check
        IF NOT EXISTS (SELECT 1 FROM Employee WHERE EmployeeID = @CheckedByEmployeeID)
        BEGIN
            ;THROW 50008, 'Employee not found.', 1;
        END

        -- Machine check (if provided)
        IF @MachineID IS NOT NULL AND NOT EXISTS (SELECT 1 FROM Machine WHERE MachineID = @MachineID)
```


```sql
        -- Machine check (if provided)
        IF @MachineID IS NOT NULL AND NOT EXISTS (SELECT 1 FROM Machine WHERE MachineID = @MachineID)
```


```sql
        BEGIN
            ;THROW 50009, 'Invalid machine ID.', 1;
        END

        -- Result check
        IF @Result NOT IN ('Pass', 'Fail')
        BEGIN
            ;THROW 50010, 'Result must be Pass or Fail.', 1;
        END

        -- Record Quality Check
        INSERT INTO QualityCheck(ProductID, MachineID, CheckDate, Result, CheckedByEmployeeID)
        VALUES(@ProductID, @MachineID, GETDATE(), @Result, @CheckedByEmployeeID);

        -- IMPORTANT: Update QualityStatus in Product table
        UPDATE Product
        SET QualityStatus = @Result
        WHERE ProductID = @ProductID;

        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;
        THROW;
    END CATCH
END;
```

Figure 53&54. *Source code of SP_RECORD_QUALITYCHECK procedure.*

```sql
USE DENE;
GO

EXEC SP_RECORD_QUALITYCHECK 2, 1, 'Pass', 1;

SELECT TOP 1 * FROM QualityCheck ORDER BY QualityCheckID DESC;
```

100 % 1 0

Sonuçlar | İletiler

	QualityCheckID	ProductID	MachineID	CheckDate	Result	CheckedByEmployeeID
1	27	2	1	2025-12-24	Pass	1

Figure 55. *New quality check record inserted into the database.*

III. Reporting Procedures

10. SP_GET_CUSTOMER_BALANCE

- **Definition:** Calculates the financial standing of a customer.
- **Logic:** Uses **CTEs (Common Table Expressions)** to aggregate total invoiced amounts and total payments separately. It calculates the RemainingBalance and determines the status ('No Debt', 'In Debt', 'Overpayment') dynamically.

```sql
DROP PROCEDURE IF EXISTS SP_GET_CUSTOMER_BALANCE;
GO --Check

CREATE PROCEDURE SP_GET_CUSTOMER_BALANCE
    @CustomerID INT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        -- Customer check
        IF NOT EXISTS (SELECT 1 FROM Customer WHERE CustomerID = @CustomerID)
        BEGIN
            THROW 50019, 'Customer not found.', 1;
        END

        -- CTE 1: Invoice summaries
        ¿WITH InvoiceSummary AS (
            SELECT
                i.CustomerID,
                COUNT(i.InvoiceID) AS TotalInvoiceCount,
                SUM(i.TotalAmount) AS TotalInvoiceAmount,
                SUM(i.TotalWithTax) AS TotalWithTax,
                AVG(i.TotalAmount) AS AverageInvoiceAmount,
                MAX(i.InvoiceDate) AS LastInvoiceDate
            FROM Invoice i
            WHERE i.CustomerID = @CustomerID
            GROUP BY i.CustomerID
        ),
        -- CTE 2: Payment summaries
        PaymentSummary AS (
            SELECT
                i.CustomerID,
                COUNT(p.PaymentID) AS TotalPayments,
                SUM(p.PaymentAmount) AS TotalPaidAmount,
                MAX(p.PaymentDate) AS LastPaymentDate
            FROM Invoice i
            INNER JOIN Payment p ON i.InvoiceID = p.InvoiceID
            WHERE i.CustomerID = @CustomerID
            GROUP BY i.CustomerID
        )
        -- Final report
        SELECT
            c.CustomerID,
            c.Name AS CustomerName,
            c.CustomerType,
            c.Email,
            ISNULL(inv.TotalInvoiceCount, 0) AS TotalInvoiceCount,
            ISNULL(inv.TotalInvoiceAmount, 0) AS TotalInvoiceAmount,
            ISNULL(inv.TotalWithTax, 0) AS TotalWithTax,
            ISNULL(inv.AverageInvoiceAmount, 0) AS AverageInvoiceAmount,
            ISNULL(pay.TotalPayments, 0) AS TotalPayments,
            ISNULL(pay.TotalPaidAmount, 0) AS TotalPaidAmount,
            ISNULL(inv.TotalWithTax, 0) - ISNULL(pay.TotalPaidAmount, 0) AS RemainingBalance,
            inv.LastInvoiceDate,
            pay.LastPaymentDate,
            CASE
                WHEN ISNULL(inv.TotalWithTax, 0) - ISNULL(pay.TotalPaidAmount, 0) = 0 THEN 'No Debt'
```

```
57                CASE
58                    WHEN ISNULL(inv.TotalWithTax, 0) - ISNULL(pay.TotalPaidAmount, 0) = 0 THEN 'No Debt'
59                    WHEN ISNULL(inv.TotalWithTax, 0) - ISNULL(pay.TotalPaidAmount, 0) > 0 THEN 'In Debt'
60                    ELSE 'Overpayment'
61                END AS PaymentStatus
62            FROM Customer c
63            LEFT JOIN InvoiceSummary inv ON c.CustomerID = inv.CustomerID
64            LEFT JOIN PaymentSummary pay ON c.CustomerID = pay.CustomerID
65            WHERE c.CustomerID = @CustomerID;
66
67        END TRY
68        BEGIN CATCH
69            THROW;
70        END CATCH
71    END;
```

Figure 56&57&58. *Source code of SP_GET_CUSTOMER_BALANCE procedure.*

```
73    EXEC SP_GET_CUSTOMER_BALANCE 1;
```

00 % 4 0 Sat: 71 Krkt: 5 BŞL CR

Sonuçlar | İletiler

	CustomerID	CustomerName	CustomerType	Email	TotalInvoiceCount	TotalInvoiceAmount	TotalWithTax	AverageInvoiceAmount	TotalPayments	TotalPaidAmount
1	1	Ali Yılmaz	Bireysel	ali.yilmaz@example.com	2	5850.00	7020.000000000	2925.000000	1	850.00

```
72
73    EXEC SP_GET_CUSTOMER_BALANCE 1;
```

100 % 4 0 Sat: 71 Krkt: 5 BŞL CRLF

Sonuçlar | İletiler

	TotalInvoiceCount	TotalInvoiceAmount	TotalWithTax	AverageInvoiceAmount	TotalPayments	TotalPaidAmount	RemainingBalance	LastInvoiceDate	LastPaymentDate	PaymentStatus
1	2	5850.00	7020.000000000	2925.000000	1	850.00	6170.00	2025-12-24	2023-01-06	In Debt

Figure 59&60. *Financial balance report output for Customer ID 1.*

11. SP_GET_PRODUCT_QUALITY_SUMMARY

- **Definition:** Provides a comprehensive quality report for products, filtered by category and pass rate.
- **Logic:**
    - Uses **Window Functions** (ROW_NUMBER) to identify the most recent checks.
    - Use STRING_AGG to concatenate recent check results into a single readable string.
    - Calculates a 'QualityGrade' (Excellent, Good, Medium, Low) based on the pass rate percentage.

```sql
DROP PROCEDURE IF EXISTS SP_GET_PRODUCT_QUALITY_SUMMARY;
GO --Check

CREATE PROCEDURE SP_GET_PRODUCT_QUALITY_SUMMARY
    @CategoryFilter NVARCHAR(50) = NULL,
    @MinPassRate DECIMAL(5,2) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        -- CTE: Top 5 recent quality checks per product
        ;WITH RecentChecks AS (
            SELECT
                qc.ProductID,
                qc.Result,
                qc.CheckDate,
                ROW_NUMBER() OVER (PARTITION BY qc.ProductID ORDER BY qc.CheckDate DESC) AS RowNum
            FROM QualityCheck qc
        ),
        -- CTE: Aggregate details of the top 5 records into a single string
        CheckDetails AS (
            SELECT
                ProductID,
                STRING_AGG(
                    CAST(Result AS NVARCHAR(10)) + ' (' + CONVERT(NVARCHAR(10), CheckDate, 120) + ')',
                    ', '
                ) WITHIN GROUP (ORDER BY CheckDate DESC) AS RecentChecksDetail
            FROM RecentChecks
```

```sql
    FROM RecentChecks
    WHERE RowNum <= 5
    GROUP BY ProductID
)
-- Main report
SELECT
    p.ProductID,
    p.ProductName,
    p.Category,
    p.Stock,
    s.SupplierName,
    COUNT(qc.QualityCheckID) AS TotalChecks,
    SUM(CASE WHEN qc.Result = 'Pass' THEN 1 ELSE 0 END) AS PassCount,
    SUM(CASE WHEN qc.Result = 'Fail' THEN 1 ELSE 0 END) AS FailCount,
    CASE
        WHEN COUNT(qc.QualityCheckID) > 0
        THEN CAST(SUM(CASE WHEN qc.Result = 'Pass' THEN 1 ELSE 0 END) * 100.0 / COUNT(qc.QualityCheckID) AS DECIMAL(5,2))
        ELSE NULL
    END AS PassRate,
    CASE
        WHEN COUNT(qc.QualityCheckID) = 0 THEN 'Unchecked'
        WHEN CAST(SUM(CASE WHEN qc.Result = 'Pass' THEN 1 ELSE 0 END) * 100.0 / COUNT(qc.QualityCheckID) AS DECIMAL(5,2)) >= 95 THEN 'Excellent'
        WHEN CAST(SUM(CASE WHEN qc.Result = 'Pass' THEN 1 ELSE 0 END) * 100.0 / COUNT(qc.QualityCheckID) AS DECIMAL(5,2)) >= 85 THEN 'Good'
        WHEN CAST(SUM(CASE WHEN qc.Result = 'Pass' THEN 1 ELSE 0 END) * 100.0 / COUNT(qc.QualityCheckID) AS DECIMAL(5,2)) >= 70 THEN 'Medium'
        ELSE 'Low'
    END AS QualityGrade,
    MAX(qc.CheckDate) AS LastCheckDate,
    cd.RecentChecksDetail -- NEW COLUMN
FROM Product p
```

```sql
        FROM Product p
        LEFT JOIN QualityCheck qc ON p.ProductID = qc.ProductID
        LEFT JOIN Supplier s ON p.SupplierID = s.SupplierID
        LEFT JOIN CheckDetails cd ON p.ProductID = cd.ProductID
        WHERE (@CategoryFilter IS NULL OR p.Category = @CategoryFilter)
        GROUP BY p.ProductID, p.ProductName, p.Category, p.Stock, s.SupplierName, cd.RecentChecksDetail
        HAVING (@MinPassRate IS NULL OR
                CAST(SUM(CASE WHEN qc.Result = 'Pass' THEN 1 ELSE 0 END) * 100.0 / NULLIF(COUNT(qc.QualityCheckID), 0)
                AS DECIMAL(5,2)) >= @MinPassRate)
        ORDER BY PassRate DESC;

    END TRY
    BEGIN CATCH
        THROW;
    END CATCH
END;
```

Figure 61&62&63. *Source code of SP_GET_PRODUCT_QUALITY_SUMMARY procedure.*

```
EXEC SP_GET_PRODUCT_QUALITY_SUMMARY;
```

Sonuçlar | İletiler

	ProductID	ProductName	Category	Stock	SupplierName	TotalChecks	PassCount	FailCount	PassRate	QualityGrade	LastCheckDate	RecentChecksDetail
1	4	Plastik Kapak P500	Plastik	300	BolTech	1	1	0	100.00	Excellent	2023-01-18	Pass (2023-01-18)
2	5	Hassas Sensör S20	Sensör	40	AsiaConnect	1	1	0	100.00	Excellent	2023-01-20	Pass (2023-01-20)
3	6	Fiber Kablo F12	Kablo	500	FiberLux	1	1	0	100.00	Excellent	2023-02-01	Pass (2023-02-01)
4	1	Metal Gövde A1	Metal Parça	120	Akçelik Tedarik	2	2	0	100.00	Excellent	2025-12-24	Pass (2025-12-24), Pass (2023-01-10)
5	2	Elektronik Kontrol Kartı K10	Elektronik	85	MegaTech Components	2	2	0	100.00	Excellent	2025-12-24	Pass (2025-12-24), Pass (2023-01-12)
6	8	Montaj Parçası MP30	Montaj	200	Mediterranean Parts	1	1	0	100.00	Excellent	2023-02-08	Pass (2023-02-08)
7	9	Robotik Kol Bileşeni RBX	Robotik	25	Future Components	1	1	0	100.00	Excellent	2023-02-10	Pass (2023-02-10)
8	14	Montaj Aparatı MA44	Montaj	180	Anadolu Elektrik	1	1	0	100.00	Excellent	2023-03-10	Pass (2023-03-10)
9	15	Servo Kontrol Ünitesi SKU90	Elektronik	55	TechLine GmbH	1	1	0	100.00	Excellent	2023-03-12	Pass (2023-03-12)
10	16	Optik Lens OL25	Optik	22	OptiParts	1	1	0	100.00	Excellent	2023-04-01	Pass (2023-04-01)
11	11	Soğutma Ünitesi S200	Soğutma	10	SinoTech Industrial	1	1	0	100.00	Excellent	2023-03-01	Pass (2023-03-01)
12	12	Polimer Kasnak PK100	Polimer	140	Future Components	1	1	0	100.00	Excellent	2023-03-03	Pass (2023-03-03)
13	18	Konveyör Zinciri KZ75	Konveyör	95	TechLine GmbH	1	1	0	100.00	Excellent	2023-04-06	Pass (2023-04-06)
14	19	Elektrik Bağlantı Kablosu E...	Kablo	400	Kuzey Parça	1	1	0	100.00	Excellent	2023-04-09	Pass (2023-04-09)
15	21	Boru Seti BS40	Metal Parça	140	Nordic Steel AB	1	1	0	100.00	Excellent	2023-05-01	Pass (2023-05-01)
16	22	Plastik Panel PP22	Plastik	160	BolTech	1	1	0	100.00	Excellent	2023-05-03	Pass (2023-05-03)
17	24	Optik Sensör OS18	Sensör	50	AsiaConnect	1	1	0	100.00	Excellent	2023-05-08	Pass (2023-05-08)
18	25	Güneş Paneli GP100	Enerji	28	SolarPro	1	1	0	100.00	Excellent	2023-05-10	Pass (2023-05-10)
19	23	Robotik Aktüatör RA50	Robotik	18	MegaSteel	1	0	1	0.00	Low	2023-05-05	Fail (2023-05-05)
20	20	Motor Vidası MV10	Vida	800	Akçelik Tedarik	1	0	1	0.00	Low	2023-04-15	Fail (2023-04-15)

Figure 64.*Comprehensive product quality summary report generated by the stored procedure.*

3.6. Triggers

One Trigger has been implemented in the system to maintain data integrity and automate maintenance processes. This trigger ensures that business rules are executed automatically without manual intervention.

1. TR_QC_FAIL_UPDATE_MACHINE

- **Target Table:** QualityCheck
- **Event:** AFTER INSERT
- **Definition:** Automatically updates the status of a machine to "UnderMaintenance" if it produces a defective product (Failed Quality Check).
- **Logic:**
  - Listens for new insertions into the QualityCheck table.
  - Checks the INSERTED virtual table to see if the Result is 'Fail'.
  - If a failure is detected, it identifies the associated MachineID.
  - Updates the Machine table:
    - Sets Status to **'UnderMaintenance'**.
    - Sets LastMaintenance date to **today**.
    - Sets NextMaintenance date to **3 months from today**.
  - Includes error handling to rollback the transaction if the update fails.
- **Business Value:** This automation ensures that faulty machines are immediately taken out of production, preventing further waste and enforcing timely maintenance.

```sql
CREATE TRIGGER TR_QC_FAIL_UPDATE_MACHINE
ON QualityCheck
AFTER INSERT
AS
BEGIN
    SET NOCOUNT ON;

    BEGIN TRY
        -- Update the status and maintenance dates of the machines
        -- associated with the failed quality checks.
        UPDATE m
        SET
            m.Status = 'UnderMaintenance',
            m.LastMaintenance = GETDATE(),
            m.NextMaintenance = DATEADD(MONTH, 3, GETDATE())
        FROM Machine m
        INNER JOIN INSERTED i
            ON m.MachineID = i.MachineID -- Joins Machine with the newly inserted failed checks
        WHERE i.Result = 'Fail'          -- Business Rule: Only trigger on failure
          AND i.MachineID IS NOT NULL;   -- Data Integrity: Ensures a valid MachineID is present

    END TRY
    BEGIN CATCH
        -- If an error occurs during the update (e.g., locking), rollback the original transaction.
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;

        -- Throw the error back to the calling application/process.
        THROW;
    END CATCH
END;
```

Figure 65&67. Source code of TR_QC_FAIL_UPDATE_MACHINE trigger.

```sql
-- Look at the status of machine 3
SELECT MachineID, Status, LastMaintenance, NextMaintenance
FROM Machine
WHERE MachineID = 3;
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	MachineID	Status	LastMaintenance	NextMaintenance
1	3	UnderMaintenance	2022-12-05	2023-02-15

Figure 68. *Machine status before the trigger execution (Status: Active).*

```sql
-- Look at the status of machine 3
SELECT MachineID, Status, LastMaintenance, NextMaintenance
FROM Machine
WHERE MachineID = 3;

-- STEP 1: Add a failed record (This action will fire the Trigger)
INSERT INTO QualityCheck (ProductID, MachineID, CheckDate, Result, CheckedByEmployeeID)
VALUES (1, 3, GETDATE(), 'Fail', 1);

-- STEP 2: Check the status of the machine again (It should now be 'UnderMaintenance')
SELECT MachineID, Status, LastMaintenance, NextMaintenance
FROM Machine
WHERE MachineID = 3;
```

100 % Sorun bulunamadı

Sonuçlar | İletiler

	MachineID	Status	LastMaintenance	NextMaintenance
1	3	UnderMaintenance	2022-12-05	2023-02-15

	MachineID	Status	LastMaintenance	NextMaintenance
1	3	UnderMaintenance	2025-12-24	2026-03-24

Figure 69. *Machine status automatically updated to 'UnderMaintenance' by the trigger after a failure report.*

4.Evolution from Conceptual Design to Physical Implementation

The transition from the Conceptual Database Design (Step 2 - ER Diagram) to the Physical Database Implementation (Step 3) involved strategic refinements to adhere strictly to **Normalization Principles (specifically 3NF)** and **Operational Efficiency**. While the initial ER diagram depicted a complex web of all potential associations, the final physical schema was structured to ensure data integrity and eliminate redundancy.

The rationale behind the differences between the two steps is detailed below:

1. Achievement of Third Normal Form (3NF)

- **Conceptual Design (Step 2):** The initial diagram contained dense connectivity, where entities were interlinked in ways that could potentially lead to **transitive dependencies**. For example, indirect relationships between *Employees*, *Partners*, and *Products* were visualized simultaneously.
- **Physical Implementation (Step 3):** To strictly satisfy **3NF**, we decomposed these relationships. We ensured that every non-key attribute in a table is dependent *only* on the Primary Key.
    - *Example:* Instead of storing redundant Supplier details within the Product entity (which would degrade the design to 2NF or 1NF), we established a strict Foreign Key (SupplierID) reference.
    - **Result:** The diagram appears "cleaner" not because it is simpler, but because **redundancy has been eliminated**, which is the primary goal of 3NF.

2. Resolution of Many-to-Many Relationships

- **Reason:** The conceptual model implied complex associations (e.g., multiple partners collaborating on multiple products) that creates ambiguity in transactional recording.
- **Refinement:** In the physical design, we prioritized direct, traceable **1:N (One-to-Many)** relationships. This ensures that every specific product batch or transaction can be traced back to a single responsible entity (Machine, Supplier, or Employee), reducing data anomalies during INSERT/UPDATE operations.

3. Separation of Business Logic from Schema Structure

- **Reason:** The ER diagram attempted to visually represent dynamic business rules (e.g., "A machine stops if quality fails").
- **Refinement:** In the physical implementation, we moved this dynamic logic out of the static table structure and into **Triggers (TR_QC_FAIL_UPDATE_MACHINE)** and **Stored Procedures**. This keeps the table schema normalized and stable, while the "complexity" is handled by the database engine's procedural code.

Conclusion: The final physical design is not a simplification of the requirements but an **optimized, 3NF-compliant realization** of the conceptual model. It guarantees data consistency, minimizes storage overhead, and supports high-performance querying as demonstrated by the implementation of Indexes and Views.

5. Conclusion

In the Step-3 phase of the Dene Technology Database Project, the **physical database implementation** has been successfully completed. The project has moved from conceptual design (Step 2) to a fully functional SQL Server environment.

Key achievements in this phase include:

- **Normalization:** All 9 tables are designed in **3rd Normal Form (3NF)**, ensuring minimal redundancy and optimal data integrity.
- **Data Integrity:** Robust use of Constraints (Check, Default, Unique) and Foreign Keys guarantees the accuracy of business data.
- **Automation:** Through **11 Stored Procedures** and **1 Trigger**, critical business rules such as machine maintenance updates upon failure and safe customer deletion are now automated at the database level.
- **Reporting:** Four complex **Views** provide real-time insights into financial and operational metrics without requiring complex ad-hoc queries.